Hadoop Programming Made Easier Admiring the Pig Architecture

- Apache Pig is an abstraction over MapReduce. It is a tool/platform which is used to analyze larger sets of data representing them as data flows. Pig is generally used with **Hadoop**; we can perform all the data manipulation operations in Hadoop using Pig.
- To write data analysis programs, Pig provides a high-level language known as **Pig Latin**. This language provides various operators using which programmers can develop their own functions for reading, writing, and processing data.
- To analyze data using **Apache Pig**, programmers need to write scripts using Pig Latin language. All these scripts are internally converted to Mapand Reduce tasks. Apache Pig has a component known as **Pig Engine** that accepts the Pig Latin scripts as input and converts those scripts into MapReduce jobs.

Why Do We Need Apache Pig?

- Programmers who are not so good at Java normally used to struggle working with Hadoop, especially while performing any MapReduce tasks. Apache Pig is a boon for all such programmers.
- Using **Pig Latin**, programmers can perform MapReduce tasks easily without having to type complex codes in Java.
- Apache Pig uses **multi-query approach**, thereby reducing the length of codes.

For example, an operation that would require you to type 200 lines of code(LoC) in Java can be easily done by typing as less as just 10 LoC in Apache Pig.

Ultimately Apache Pig reduces the development time by almost 16 times.

- Pig Latin is **SQL-like language** and it is easy to learn Apache Pig when you are familiar with SQL.
- Apache Pig provides many built-in operators to support data operations like joins, filters, ordering, etc. In addition, it also provides nested data typeslike tuples, bags, and maps that are missing from MapReduce.

Apache Pig Properties

It has following key properties such as:

- **Ease of programming**

Basically, when all the complex tasks comprised of multiple interrelated data transformations are explicitly encoded as data flow sequences, that makes them easy to write, understand, and maintain.

- **Optimization opportunities**

It allows users to focus on semantics rather than efficiency, to optimize their execution automatically, in which tasks are encoded permits the system.

- **Extensibility**

In order to do special-purpose processing, users can create their own functions. Hence, programmers need to write scripts using Pig Latin language to analyze data using Apache Pig.

Apache Pig comes with the following features −

- **Rich set of operators** − It provides many operators to perform operationslike join, sort, filer, etc.
- **Ease of programming** − Pig Latin is similar to SQL and it is easy to write aPig script if you are good at SQL.
- **Optimization opportunities** − The tasks in Apache Pig optimize theirexecution automatically, so the programmers need to focus only onsemantics of the language.
- **Extensibility** − Using the existing operators, users can develop their ownfunctions to read, process, and write data.
- **UDF's** − Pig provides the facility to create **User-defined Functions** in otherprogramming languages such as Java and invoke or embed them in Pig Scripts.
- **Handles all kinds of data** − Apache Pig analyses all kinds of data, bothstructured as well as unstructured. It stores the results in HDFS.

Why Do We Need Apache Pig?

While performing any MapReduce tasks, there is a case Programmers who are not so good at Java normally used to struggle to work with Hadoop. Thus, we can say, Pig is a boon for all such programmers because:

- Without having to type complex codes in Java, using Pig Latin, programmers can perform MapReduce tasks easily.
- It also helps in reduce the length of codes, since Pig uses multi-query approach. Let's understand it with an example. Here an operation that would require us to type 200 lines of code (LoC) in Java can be easily done by typing as less as just 10 LoC in Apache Pig. Hence, it shows, Pig reduces the development time by almost 16 times.
- When you are familiar with SQL, it is easy to learn Pig. Because Pig Latin is SQL-like language.
- It offers many built-in operators, in order to support data operations such as joins, filters, ordering, and many more. Also, it offers nested data types that are missing from MapReduce such as tuples, bags, and maps

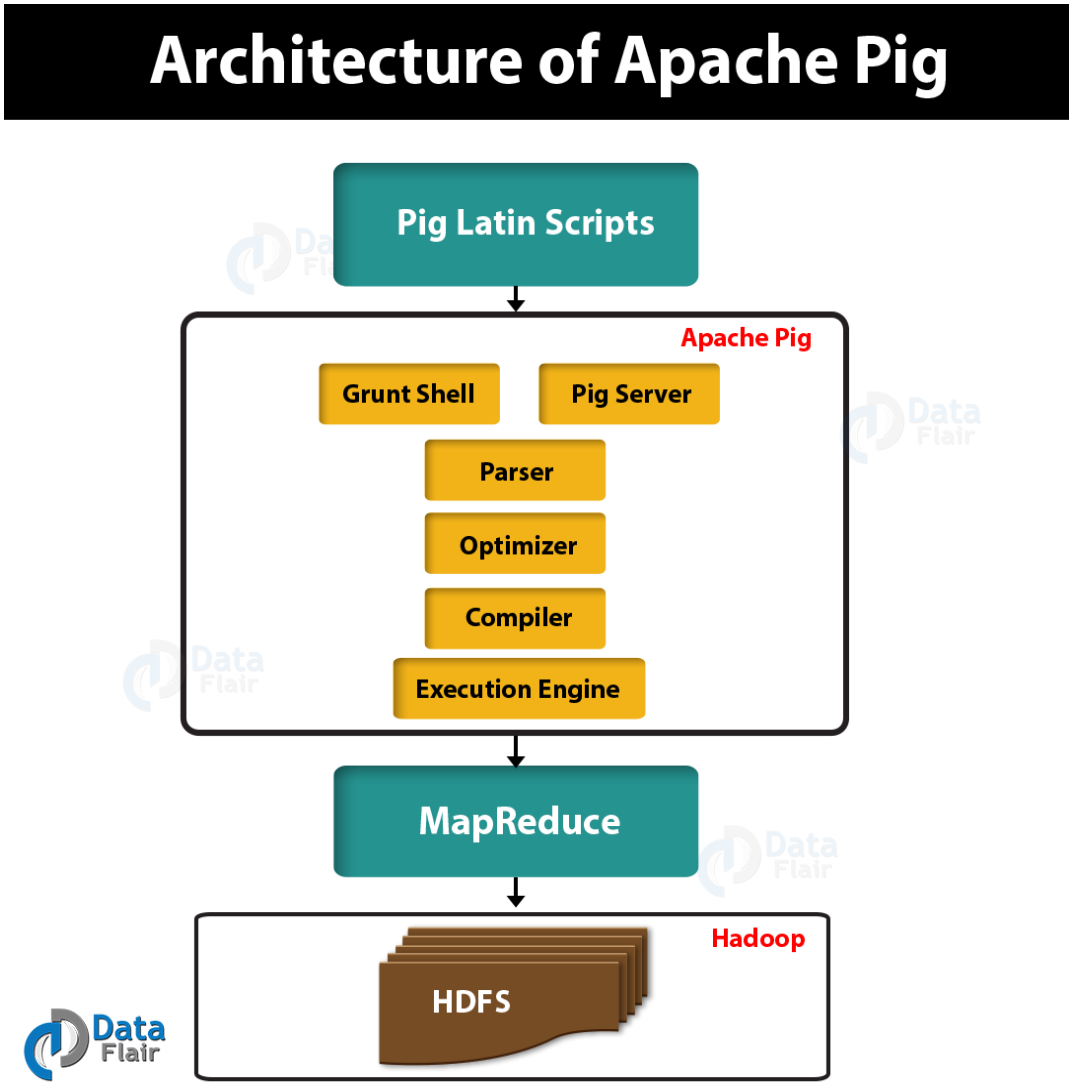
There are several scenarios, where we can use Pig. Such as:

- While data loads are time sensitive.
- Also, while processing various data sources.
- While we require analytical insights through sampling.

Where Not to Use Pig?

Also, there are some Scenarios, where we cannot use. Such as:

- While the data is completely unstructured. Such as video, audio, and readable text.
- Where time constraints exist. Since Pig is slower than MapReduce jobs.
- Also, when more power is required to optimize the codes, we cannot use Pig.



For writing a Pig script, we need a Pig Latin language, and to execute them, we need an execution environment. First, we will start from Pig Latin Script.

1. Pig Latin Scripts

Initially, we submit Pig scripts to the Pig execution environment which can be written in Pig latin code using built-in operators. There are three different ways to execute the Pig script:

Grunt Shell: This is Pig's interactive shell, where we generally execute all Pig commands/scripts.

Script File: We write all the Pig commands in a script file using the editor(nano/vi/gedit) and execute the Pig script file from the local file system. This is executed by the help of the Pig server.

Embedded Script: This is, in fact, a way to other language's capability. We can create user defined functions to bring the functionality which is not available in built-in operators, using

different languages like Java, Python, Ruby etc. and embed it in Pig Latin script file. Then, execute that script file.

2. Parser

In this phase, Pig scripts are given to the parser, and it does type checking and checks the syntax of the pig script. The outcome of parser is a DAG(Directed Acyclic graph), which represents the Pig latin statements and logical operators. The data flows are represented as edges, and logical operators are represented as the nodes.

3.Optimizer

From the parser, the DAG is submitted to the optimizer to perform automatic optimization activities like split, merge, transform, and reorder operators, etc. The optimizer aims to reduce the amount of data in the pipeline at any instance of time while processing the extracted data. It uses the below functions to perform these activities.

PushUpFilter: If there are multiple conditions used in the filter, and the filter can be split, Apache Pig splits the conditions and pushes up each condition separately. Early selection of these conditions helps in reducing the number of data records remaining in the pipeline.

PushDownForEachFlatten: Flattens produces a cross product between a tuple or a bag and other fields in the data record, as late as possible in the plan. It helps to keep the number of records low in the pipeline.

ColumnPruner: Omitting the unused or no longer needed columns, reduce the size of the record. This can be applied for each operator so that fields can be pruned as aggressively as possible.

MapKeyPruner: This function omits never used map keys, which reduce the size of the record.

LimitOptimizer: If the limit operators are immediately applied after a sort or load operator, Pig converts the sort or sort operator into a limit-sensitive implementation, which does not require processing the whole dataset. Applying the limit at an earlier phase reduces the number of records.

4. Compiler

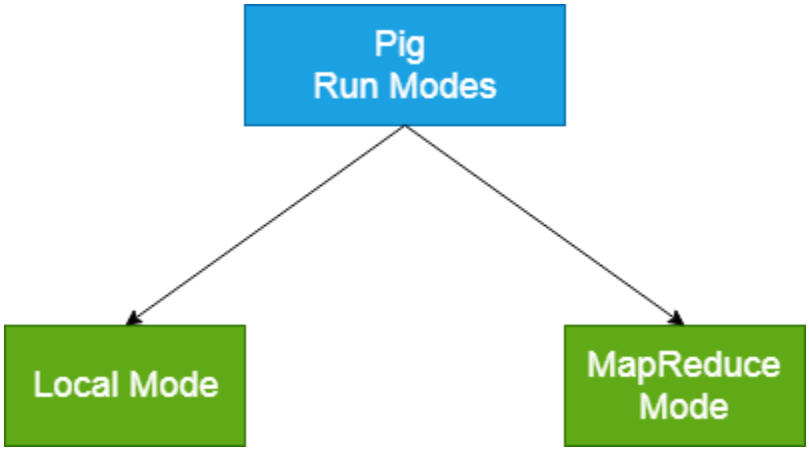
After finishing the optimization, the compiler compiles the optimized code into a series of MapReduce jobs The compiler is responsible for converting Pig jobs automatically into MapReduce jobs.

5. Execution engine

Finally, these MR jobs are submitted for execution to the execution engine Then the MR jobs are executed and give the required outcome. This result can be displayed on the screen using the Dump statement and can be stored in the HDFS using the store statement.

Apache Pig Run Modes

Apache Pig executes in two modes: Local Mode and MapReduce Mode.



Local Mode

- All scripts are run on a single machine without requiring Hadoop MapReduce and HDFS.
- It executes in a single JVM and is used for development experimenting and prototyping.
- Here, files are installed and run using localhost.
- The local mode works on a local file system. The input and output data stored in the local file system.

The command for local mode grunt shell:

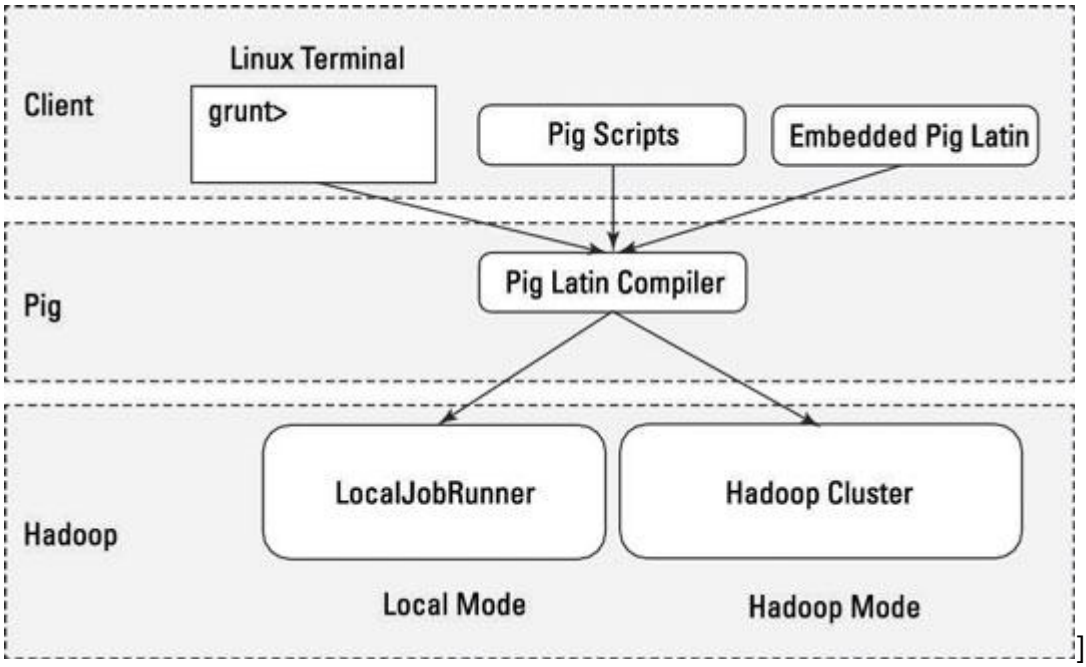
```
$ pig-x local
```

MapReduce Mode

- The MapReduce mode is also known as Hadoop Mode.
- It is the default mode.
- In this Pig renders Pig Latin into MapReduce jobs and executes them on the cluster.
- It can be executed against semi-distributed or fully distributed Hadoop installation.
- Here, the input and output data are present on HDFS.

The command for Map reduce mode:

$ pig -x mapreduce



Ways to execute Pig Program

These are the following ways of executing a Pig program on local and MapReduce mode: -

- **Interactive Mode** - In this mode, the Pig is executed in the Grunt shell. To invoke Grunt shell, run the pig command. Once the Grunt mode executes, we can provide Pig Latin statements and command interactively at the command line.
- **Batch Mode** - In this mode, we can run a script file having a .pig extension. These files contain Pig Latin commands.
- **Embedded Mode** - In this mode, we can define our own functions. These functions can be called as UDF (User Defined Functions). Here, we use programming languages like Java and Python.

Pig Script Interfaces in Hadoop

The Pig programming language is designed to handle any kind of data tossed its way — structured, semi-structured, unstructured data, you name it. Pig programs can be packaged in three different ways:

- **Script:** This method is nothing more than a file containing Pig Latin commands, identified by the .pig suffix (FlightData.pig, for example). Ending your Pig program with the .pig extension is a convention but not required. The commands are interpreted by the Pig Latin compiler and executed in the order determined by the Pig optimizer.

- **Grunt:** Grunt acts as a command interpreter where you can interactively enter Pig Latin at the Grunt command line and immediately see the response. This method is helpful for prototyping during initial development and with what-if scenarios.

- **Embedded:** Pig Latin statements can be executed within Java, Python, or JavaScript programs.

Pig scripts, Grunt shell Pig commands, and embedded Pig programs can run in either Local mode or MapReduce mode.

The Grunt shell provides an interactive shell to submit Pig commands or run Pig scripts. To start the Grunt shell in Interactive mode, just submit the command pig at your shell.

To specify whether a script or Grunt shell is executed locally or in Hadoop mode just specify it in the –x flag to the pig command. The following is an example of how you'd specify running your Pig script in local mode

```
pig –x local milesPerCarrier.pig
```

Here's how you'd run the pig script in Hadoop mode, which is the default if you don't specify the flag:

```
pig –x mapreducemilesPerCarrier.pig
```

Pig Latin

The Pig Latin is a data flow language used by Apache Pig to analyze the data in Hadoop. It is a textual language that abstracts the programming from the Java MapReduce idiom into a notation.

Pig Latin Statements

The Pig Latin statements are used to process the data. It is an operator that accepts a relation as an input and generates another relation as an output.

- It can span multiple lines.
- Each statement must end with a semi-colon.
- It may include expression and schemas.
- By default, these statements are processed using multi-query execution

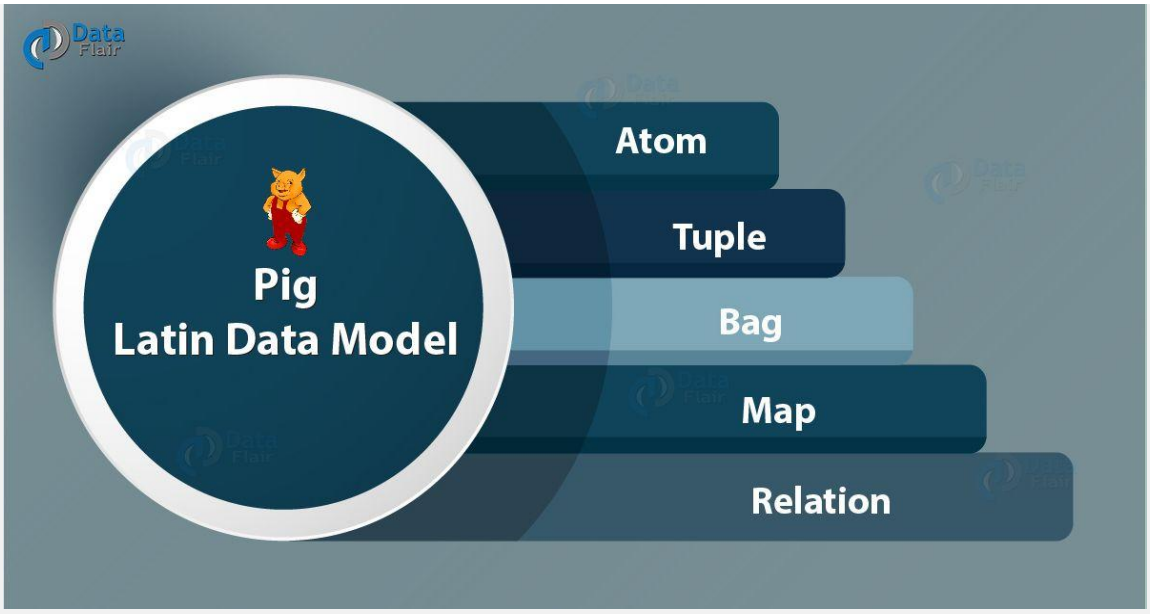
Latin Data Types

Simple Data Types

Type	Description
int	It defines the signed 32-bit integer. Example - 2
long	It defines the signed 64-bit integer. Example - 2L or 2l
float	It defines 32-bit floating point number. Example - 2.5F or 2.5f or 2.5e2f or 2.5.E2F
double	It defines 64-bit floating point number. Example - 2.5 or 2.5 or 2.5e2f or 2.5.E2F
chararray	It defines character array in Unicode UTF-8 format. Example - javatpoint

bytearray	It defines the byte array.
boolean	It defines the boolean type values. Example - true/false
datetime	It defines the values in datetime order. Example - 1970-01- 01T00:00:00.000+00:00
biginteger	It defines Java BigInteger values. Example - 5000000000000
bigdecimal	It defines Java BigDecimal values. Example - 52.232344535345

Pig Latin Data Model



Apache Pig architecture – Pig Latin Data Model

Pig Latin data model is fully nested. Also, it allows complex non-atomic data types like map and tuple.

i. Atom

Atom is defined as any single value in Pig Latin, irrespective of their data. Basically, we can use it as string and number and store it as the string. Atomic values of Pig are int, long, float, double, char array, and byte array. Moreover, a field is a piece of data or a simple atomic value in Pig.

For Example − ‘Shubham’ or ‘25’

ii. Tuple

Tuple is a record that is formed by an ordered set of fields. However, the fields can be of any type. In addition, a tuple is similar to a row in a table of RDBMS.

For Example − (Shubham, 25)

iii. Bag

An unordered set of tuples is what we call Bag. To be more specific, a Bag is a collection of tuples (non-unique). Moreover, each tuple can have any number of fields (flexible schema). Generally, we represent a bag by '{}'.
For Example − {(Shubham, 25), (Pulkit, 35)}
In addition, when a bag is a field in a relation, in that way it is known as the inner bag. Example − {Shubham, 25, {9826022258, Shubham@gmail.com,}}

iv. Map

A set of key-value pairs is what we call a map (or data map). Basically, the key needs to be of type char array and should be unique. Also, the value might be of any type. And, we represent it by '[ ]'
For Example − [name#Shubham, age#25]

v. Relation

A bag of tuples is what we call Relation. In Pig Latin, the relations are unordered. Also, there is no guarantee that tuples are processed in any particular order.
So, this was all in Apache Pig Architecture.

Pig Latin – Statements

While processing data using Pig Latin, **statements** are the basic constructs.

- These statements work with **relations**.
- They include **expressions** and **schemas**.
- Every statement ends with a semicolon (;).
- We will perform various operations using operators provided by Pig Latin, through statements.
- Except LOAD and STORE, while performing all other operations, Pig Latin statements take a relation as input and produce another relation as output.
- As soon as you enter a **Load** statement in the Grunt shell, its semantic checking will be carried out. To see the contents of the schema, you need to use the **Dump** operator. Only after performing the **Dump** operation, the MapReduce job for loading the data into the file system will be carried out.

Pig Latin – Data types

Given below table describes the Pig Latin data types.

S.N.	Data Type	Description & Example
1	int	Represents a signed 32-bit integer. **Example** : 8
2	long	Represents a signed 64-bit integer. **Example** : 5L
3	float	Represents a signed 32-bit floating point. **Example** : 5.5F
4	double	Represents a 64-bit floating point. **Example** : 10.5
5	chararray	Represents a character array (string) in Unicode UTF-8 format. **Example** : ‘tutorials point’
6	Bytearray	Represents a Byte array (blob).
7	Boolean	Represents a Boolean value. **Example** : true/ false.
8	Datetime	Represents a date-time. **Example** : 1970-01-01T00:00:00.000+00:00
9	Biginteger	Represents a Java BigInteger. **Example** : 60708090709

10	Bigdecimal	Represents a Java BigDecimal **Example** : 185.98376256272893883
Complex Types		
11	Tuple	A tuple is an ordered set of fields. **Example** : (raja, 30)
12	Bag	A bag is a collection of tuples. **Example** : {(raju,30),(Mohhammad,45)}
13	Map	A Map is a set of key-value pairs. **Example** : [‘name’#’Raju’, ‘age’#30]

Null Values

Values for all the above data types can be NULL. Apache Pig treats null values in a similar way as SQL does.

A null can be an unknown value or a non-existent value. It is used as a placeholder for optional values. These nulls can occur naturally or can be the result of an operation.

Pig Latin – Arithmetic Operators

The following table describes the arithmetic operators of Pig Latin. Suppose a = 10 and b = 20.

Operator	Description	Example
+	**Addition** − Adds values on either side of the operator	a + b will give 30
−	**Subtraction** − Subtracts right hand operand from left hand operand	a − b will give −10
*	**Multiplication** − Multiplies values on either side of the operator	a * b will give 200
/	**Division** − Divides left hand operand by right hand operand	b / a will give 2
%	**Modulus** − Divides left hand operand by right hand operand and returns remainder	b % a will give 0
? :	**Bincond** − Evaluates the Boolean operators. It has three operands as shown below. variable **x** = (expression) ? **value1** *if true* : **value2** *if false*.	b = (a == 1)? 20: 30; if a = 1 the value of b is 20. if a!=1 the value of b is 30.
CASE WHEN THEN ELSE END	**Case** − The case operator is equivalent to nested bincond operator.	CASE f2 % 2 WHEN 0 THEN 'even' WHEN 1 THEN 'odd' END

Pig Latin – Comparison Operators

The following table describes the comparison operators of Pig Latin.

Operator	Description	Example
==	**Equal** − Checks if the values of two operands are equal or not; if yes, then the condition becomes true.	(a = b) is not true
!=	**Not Equal** − Checks if the values of two operands are equal or not. If the values are not equal, then condition becomes true.	(a != b) is true.
>	**Greater than** − Checks if the value of the left operand is greater than the value of the right operand. If yes, then the condition becomes true.	(a > b) is not true.
<	**Less than** − Checks if the value of the left operand is less than the value of the right operand. If yes, then the condition becomes true.	(a < b) is true.
>=	**Greater than or equal to** − Checks if the value of the left operand is greater than or equal to the value of the right operand. If yes, then the condition becomes true.	(a >= b) is not true.
<=	**Less than or equal to** − Checks if the value of the left operand is less than or equal to the value of the right operand. If yes, then the condition becomes true.	(a <= b) is true.
matches	**Pattern matching** − Checks whether the string in the left-hand side matches with the constant in the right-hand side.	f1 matches '.*tutorial.*'

Pig Latin – Type Construction Operators

The following table describes the Type construction operators of Pig Latin.

Operator	Description	Example
()	**Tuple constructor operator** − This operator is used to construct a tuple.	(Raju, 30)
{}	**Bag constructor operator** − This operator is used to construct a bag.	{(Raju, 30), (Mohammad, 45)}
[]	**Map constructor operator** − This operator is used to construct a tuple.	[name#Raja, age#30]

Pig Latin – Relational Operations

The following table describes the relational operators of Pig Latin.

Operator	Description
Loading and Storing	
LOAD	To Load the data from the file system (local/HDFS) into a relation.
STORE	To save a relation to the file system (local/HDFS).
Filtering	
FILTER	To remove unwanted rows from a relation.
DISTINCT	To remove duplicate rows from a relation.
FOREACH, GENERATE	To generate data transformations based on columns of data.
STREAM	To transform a relation using an external program.
Grouping and Joining	
JOIN	To join two or more relations.
COGROUP	To group the data in two or more relations.
GROUP	To group the data in a single relation.

CROSS	To create the cross product of two or more relations.
Sorting	
ORDER	To arrange a relation in a sorted order based on one or more fields (ascending or descending).
LIMIT	To get a limited number of tuples from a relation.
Combining and Splitting	
UNION	To combine two or more relations into a single relation.
SPLIT	To split a single relation into two or more relations.
Diagnostic Operators	
DUMP	To print the contents of a relation on the console.
DESCRIBE	To describe the schema of a relation.
EXPLAIN	To view the logical, physical, or MapReduce execution plans to compute a relation.
ILLUSTRATE	To view the step-by-step execution of a series of statements.

Apache Pig LOAD Operator

The Apache Pig LOAD operator is used to load the data from the file system.

Syntax

LOAD 'info' [USING FUNCTION] [AS SCHEMA];

Here,

- **LOAD** is a relational operator.
- **'info'** is a file that is required to load. It contains any type of data.
- **USING** is a keyword.
- **FUNCTION** is a load function.
- **AS** is a keyword.
- **SCHEMA** is a schema of passing file, enclosed in parentheses.

Functions

- Pigstorage(delimiter) -> for structured files(default)
- textloader, (unstructured files)
- binstorage,(machine readable format)
- jsonstorage(json (java script object notation)format)

Schema

columnname : data type

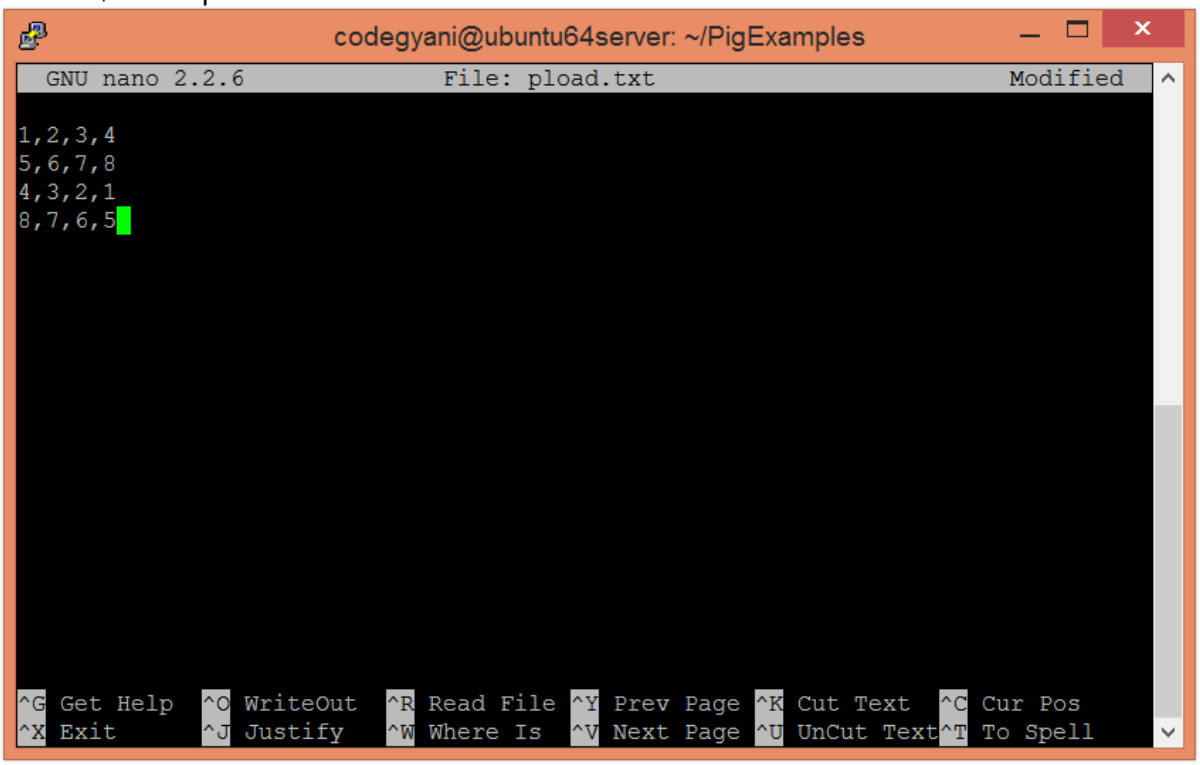
→ default column names $01, $02,....

→ default type is bytearray

Steps to execute LOAD Operator

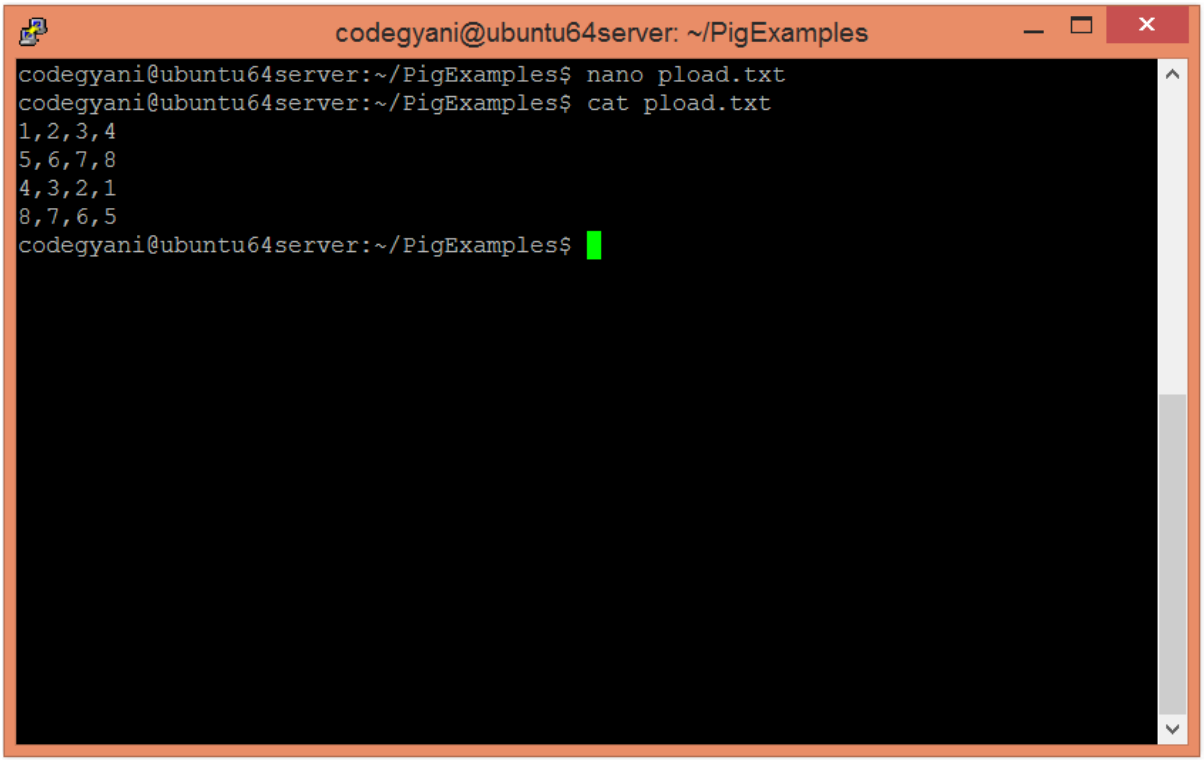
- Create a text file in your local machine and provide some values to it.

1. $ nano pload.txt



- Check the values written in the text files.

1. $ cat pload.txt



- Upload the text files on HDFS in the specific directory.

```
$ hdfs dfs -put pload.txt /pigexample
```

- Open the pig MapReduce run mode.

```
$ pig
```

- Load the file that contains the data.

```
grunt> A = LOAD '/pigexample/pload.txt' USING PigStorage(',')
                    AS (a1:int,a2:int,a3:int,a4:int) ;
```

- Now, execute and verify the data.

```
grunt> DUMP A;
```

```
codegyani@ubuntu64server: ~/PigExamples
aggregation.
2019-02-25 07:18:49,966 [main] INFO  org.apache.pig.backend.hadoop.executionengi
ne.mapReduceLayer.MapReduceLauncher - Success!
2019-02-25 07:18:50,007 [main] WARN  org.apache.pig.data.SchemaTupleBackend - Sc
hemaTupleBackend has already been initialized
2019-02-25 07:18:50,176 [main] INFO  org.apache.hadoop.mapreduce.lib.input.FileI
nputFormat - Total input paths to process : 1
2019-02-25 07:18:50,181 [main] INFO  org.apache.pig.backend.hadoop.executionengi
ne.util.MapRedUtil - Total input paths to process : 1
(1,2,3,4)
(5,6,7,8)
(4,3,2,1)
(8,7,6,5)
grunt>
```

- Let's check the corresponding schema.

1. grunt> DESCRIBE A;

```
codegyani@ubuntu64server: ~/PigExamples
grunt> DESCRIBE A;
A: {a1: int,a2: int,a3: int,a4: int}
grunt>
```

Given below is the syntax of **TextLoader()** function.

grunt> TextLoader()

Example

Let us assume there is a file with named **stu_data.txt** in the HDFS directory named ***/data/*** as shown below.

001,Rajiv_Reddy,21,Hyderabad
002,siddarth_Battacharya,22,Kolkata
003,Rajesh_Khanna,22,Delhi
004,Preethi_Agarwal,21,Pune
005,Trupthi_Mohanthy,23,Bhuwaneshwar
006,Archana_Mishra,23,Chennai
007,Komal_Nayak,24,trivendram
008,Bharathi_Nambiayar,24,Chennai

Now let us load the above file using the **TextLoader()** function.

```
grunt> details = LOAD 'hdfs://localhost:9000/pig_data/stu_data.txt' USING
TextLoader();
```

You can verify the loaded data using the Dump operator.

grunt> dump details;

(001,Rajiv_Reddy,21,Hyderabad)
(002,siddarth_Battacharya,22,Kolkata)
(003,Rajesh_Khanna,22,Delhi)
(004,Preethi_Agarwal,21,Pune)
(005,Trupthi_Mohanthy,23,Bhuwaneshwar)
(006,Archana_Mishra,23,Chennai)
(007,Komal_Nayak,24,trivendram)
(008,Bharathi_Nambiayar,24,Chennai)

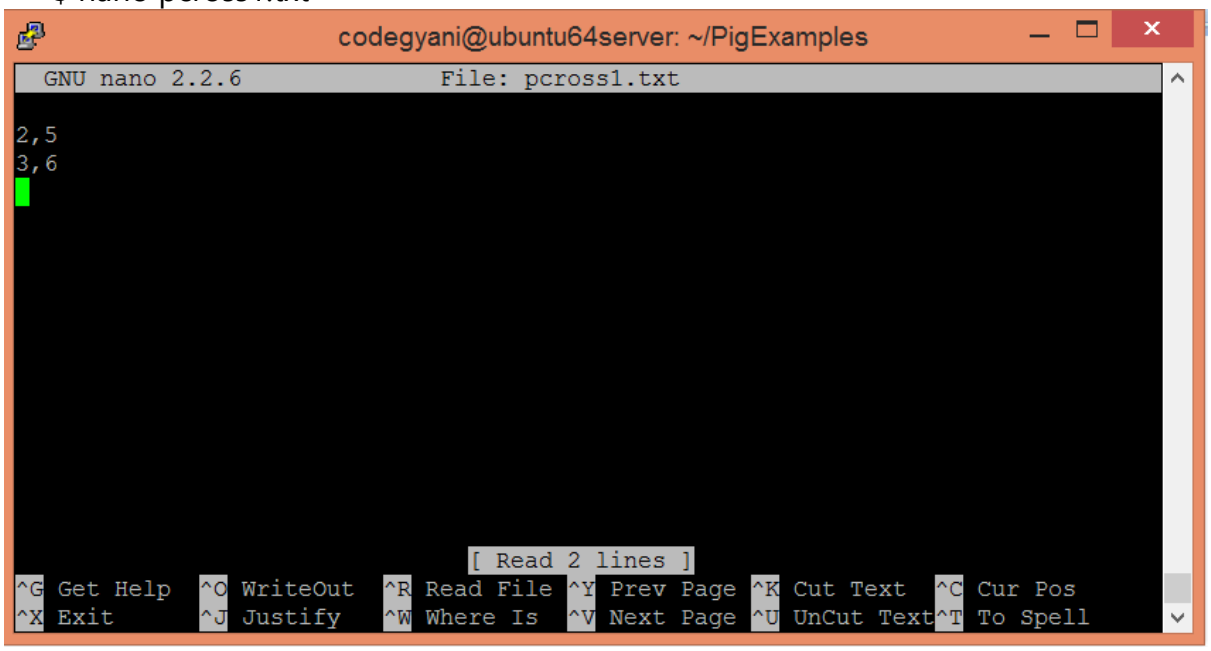
Apache Pig CROSS Operator

The Apache Pig CROSS operator facilitates to compute the cross product of two or more relations. Using CROSS operator is an expensive operation and should be used sparingly.

Steps to execute CROSS Operator

- Create a text file in your local machine and write some values into it.

$ nano pcross1.txt



- Create another text file in your local machine and also write some values into it.

$ nano pcross2.txt

- Check the values written in both text files.

$ cat pcross1.txt

$ cat pcross2.txt

- Upload both text files on HDFS in the specific directory.

$ hdfs dfs -put pcross1.txt /pigexample

$ hdfs dfs -put pcross2.txt /pigexample

- Open the pig MapReduce run mode.

$ pig

- Load the file that contains the data.

grunt> A = LOAD '/pigexample/pcross1.txt' USING PigStorage(',')
AS (a1:int,a2:int);

- Now, execute and verify the data.

grunt> DUMP A;

- Load the another file that contains the data.

grunt> B = LOAD '/pigexample/pcross2.txt' USING PigStorage(',')
AS (b1:int,b2:int,b3:int);

- Now, execute and verify the data.

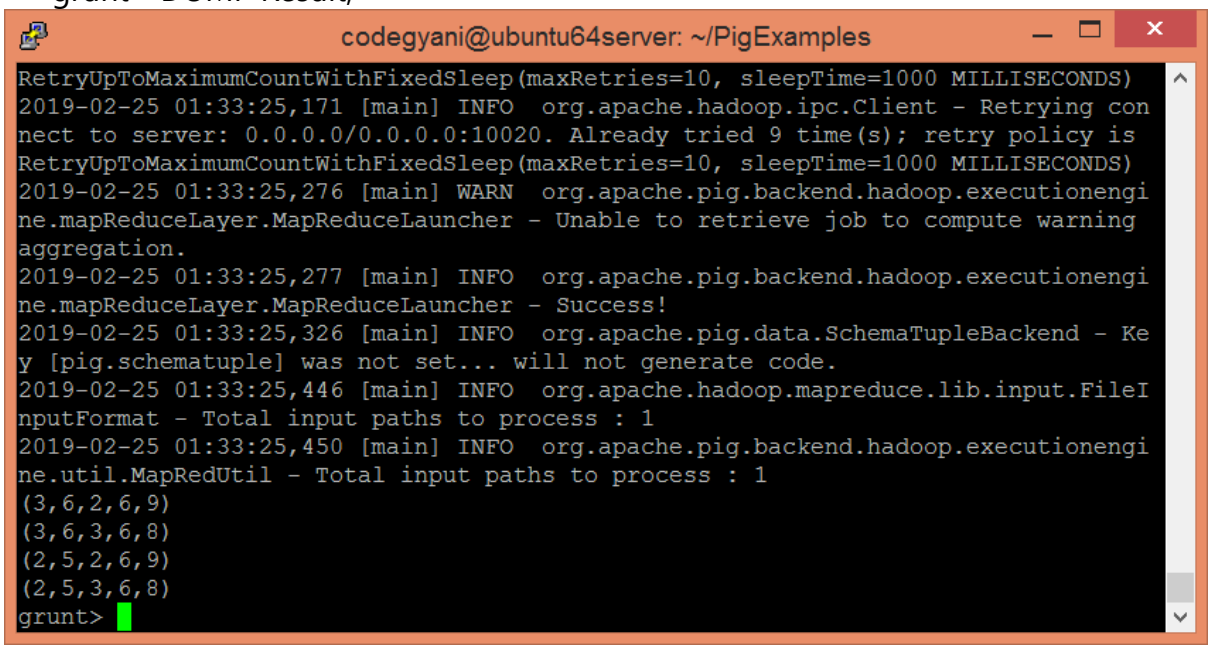
grunt> DUMP B;

- Let's perform the Cartesian product between both files.

grunt> Result = CROSS A,B;

- Now, execute and verify the data.

grunt> DUMP Result;



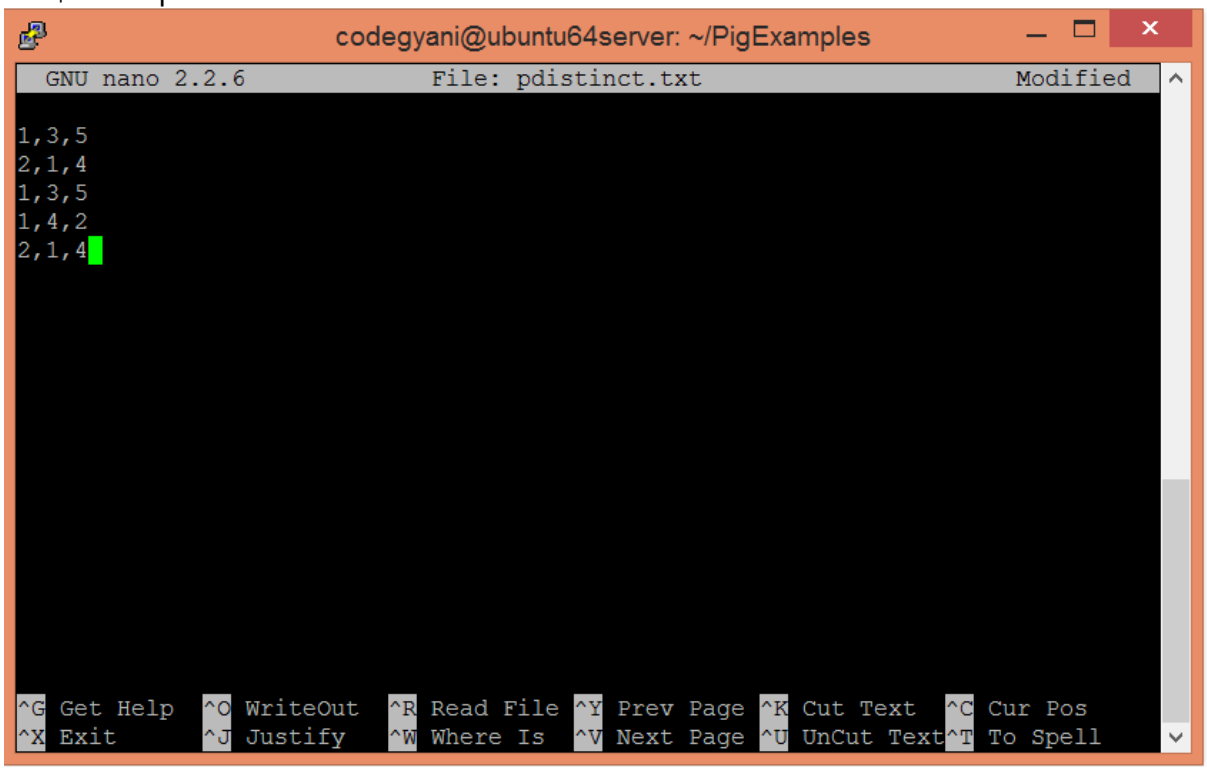
Apache Pig DISTINCT Operator

The Apache Pig DISTINCT operator is used to remove duplicate tuples in a relation. Initially, Pig sorts the given data and then eliminates duplicates.

Steps to execute DISTINCT Operator

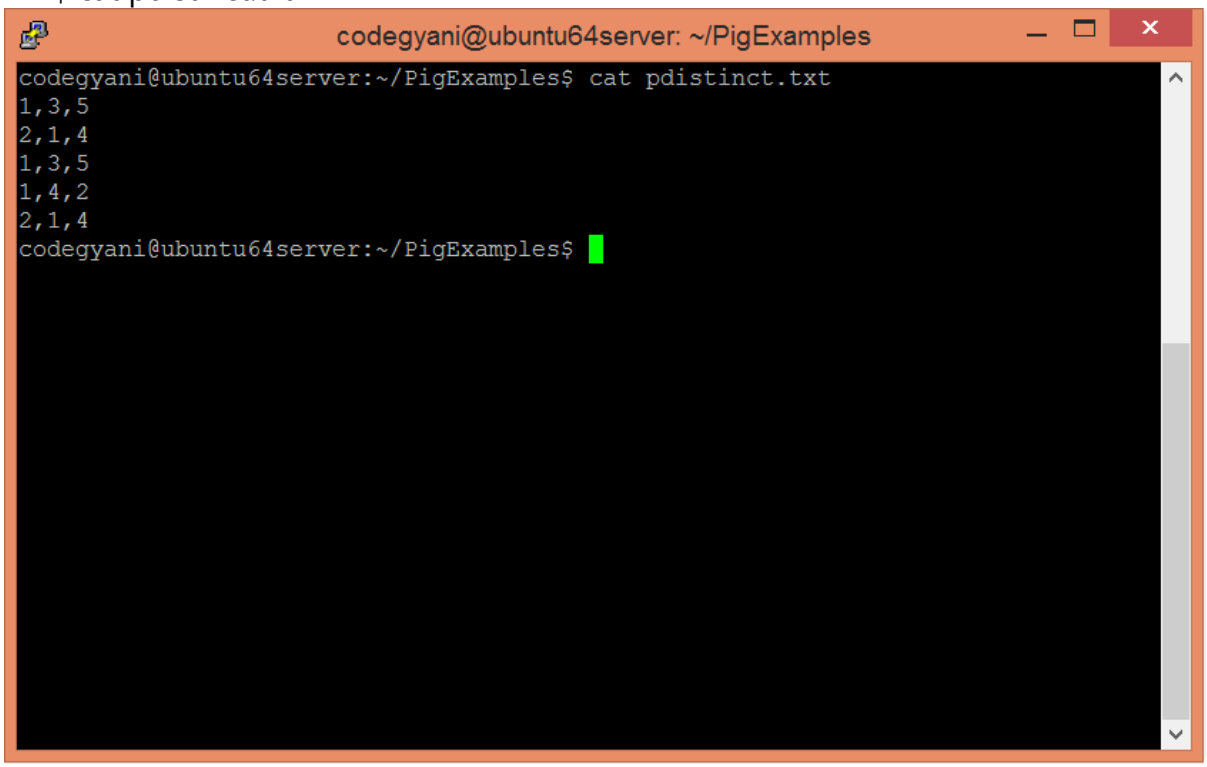
- Create a text file in your local machine and provide some values to it.

$ nano pdistinct.txt



- Check the values written in the text files.

$ cat pdistinct.txt



- Upload the text files on HDFS in the specific directory.

$ hdfs dfs -put pdistinct.txt /pigexample

- Open the pig MapReduce run mode.

$ pig

- Load the file that contains the data.

```
grunt> A = LOAD '/pigexample/pdistinct.txt' USING PigStorage(',')
                    as (a1:int,a2:int,a3:int);
```

- Now, execute and verify the data.

```
grunt> DUMP A;
```

```
codegyani@ubuntu64server: ~/PigExamples
aggregation.
2019-02-25 04:44:22,669 [main] INFO  org.apache.pig.backend.hadoop.executionengi
ne.mapReduceLayer.MapReduceLauncher - Success!
2019-02-25 04:44:22,698 [main] INFO  org.apache.pig.data.SchemaTupleBackend - Ke
y [pig.schematuple] was not set... will not generate code.
2019-02-25 04:44:22,853 [main] INFO  org.apache.hadoop.mapreduce.lib.input.FileI
nputFormat - Total input paths to process : 1
2019-02-25 04:44:22,857 [main] INFO  org.apache.pig.backend.hadoop.executionengi
ne.util.MapRedUtil - Total input paths to process : 1
(1,3,5)
(2,1,4)
(1,3,5)
(1,4,2)
(2,1,4)
grunt>
```

- Let's execute DISTINCT operator to eliminate duplicate tuples.

grunt> Result = DISTINCT A;

- Now, execute and verify the data.

grunt> DUMP Result;

```
codegyani@ubuntu64server: ~/PigExamples
2019-02-25 04:53:14,595 [main] WARN  org.apache.pig.backend.hadoop.executionengi
ne.mapReduceLayer.MapReduceLauncher - Unable to retrieve job to compute warning
aggregation.
2019-02-25 04:53:14,598 [main] INFO  org.apache.pig.backend.hadoop.executionengi
ne.mapReduceLayer.MapReduceLauncher - Success!
2019-02-25 04:53:14,601 [main] INFO  org.apache.pig.data.SchemaTupleBackend - Ke
y [pig.schematuple] was not set... will not generate code.
2019-02-25 04:53:14,694 [main] INFO  org.apache.hadoop.mapreduce.lib.input.FileI
nputFormat - Total input paths to process : 1
2019-02-25 04:53:14,696 [main] INFO  org.apache.pig.backend.hadoop.executionengi
ne.util.MapRedUtil - Total input paths to process : 1
(1,3,5)
(1,4,2)
(2,1,4)
grunt>
```

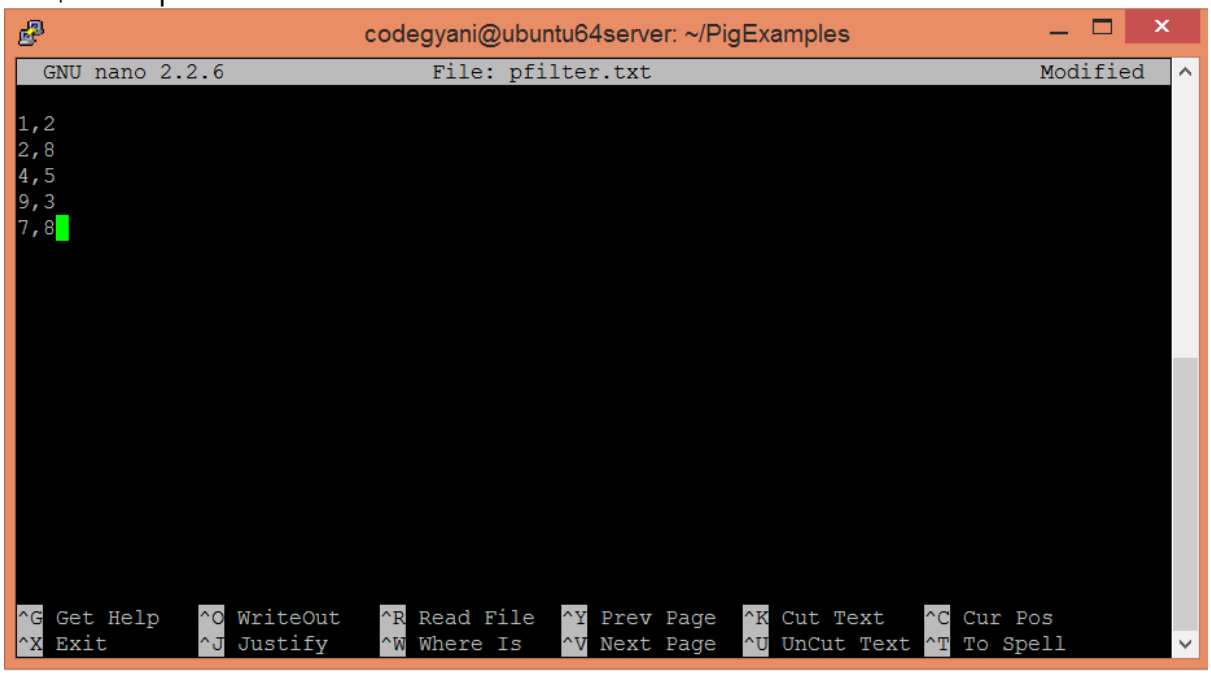
Apache Pig FILTER Operator

The Apache Pig FILTER operator is used to remove duplicate tuples in a relation. Initially, Pig sorts the given data and then eliminates duplicates.

Steps to execute FILTER Operator

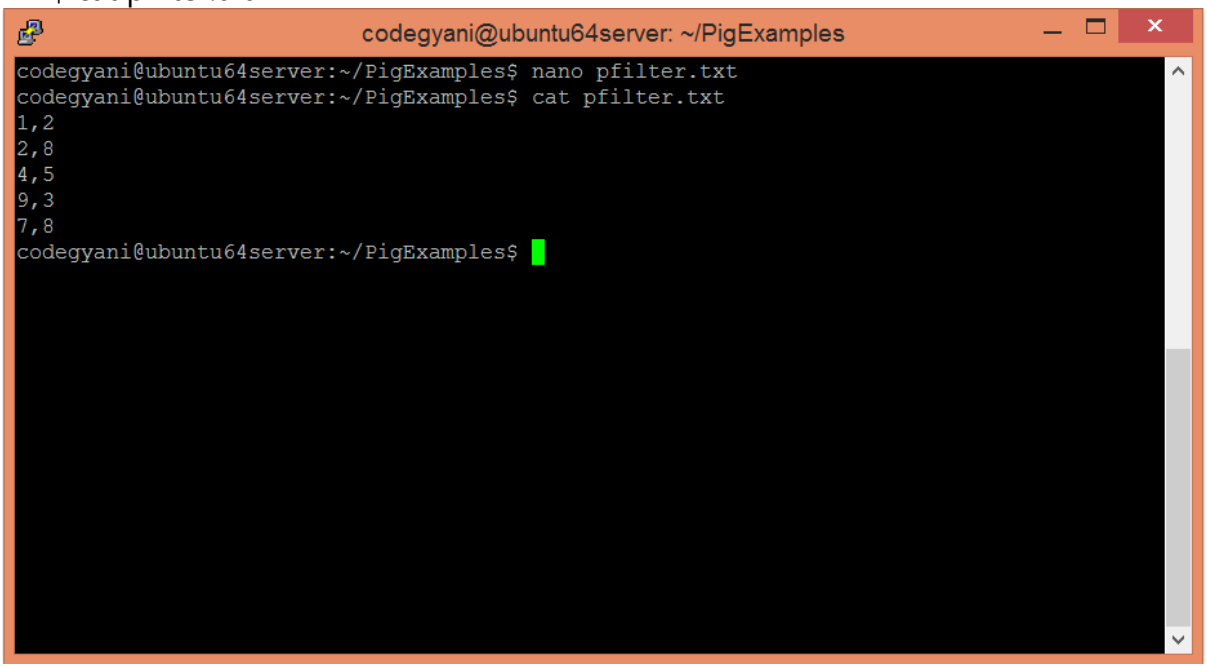
- Create a text file in your local machine and provide some values to it.

$ nano pfilter.txt



- Check the values written in the text files.

$ cat pfilter.txt



- Upload the text files on HDFS in the specific directory.

$ hdfs dfs -put pfilter.txt /pigexample

- Open the pig MapReduce run mode.

$ pig

- Load the file that contains the data.

grunt> A = LOAD '/pigexample/pfilter.txt' USING PigStorage(',') AS (a1:int,a2:int);

- Now, execute and verify the data

grunt> DUMP A;

```
codegyani@ubuntu64server: ~/PigExamples
2019-02-25 06:13:20,029 [main] INFO  org.apache.pig.backend.hadoop.executionengine.map
uceLayer.MapReduceLauncher - Success!
2019-02-25 06:13:20,064 [main] INFO  org.apache.pig.data.SchemaTupleBackend - Key [pig
hematuple] was not set... will not generate code.
2019-02-25 06:13:20,258 [main] INFO  org.apache.hadoop.mapreduce.lib.input.FileInputFo
t - Total input paths to process : 1
2019-02-25 06:13:20,263 [main] INFO  org.apache.pig.backend.hadoop.executionengine.uti
apRedUtil - Total input paths to process : 1
(1,2)
(2,8)
(4,5)
(9,3)
(7,8)
grunt>
```

- Let's execute FILTER operator to eliminate duplicate tuples.

grunt> Result = FILTER A BY a2==8;

- Now, execute and verify the data.

grunt> DUMP Result;

```
codegyani@ubuntu64server: ~/PigExamples
2019-02-25 06:24:50,721 [main] INFO  org.apache.hadoop.ipc.Client - Retrying connect t
o server: 0.0.0.0/0.0.0.0:10020. Already tried 9 time(s); retry policy is RetryUpToMax
imumCountWithFixedSleep(maxRetries=10, sleepTime=1000 MILLISECONDS)
2019-02-25 06:24:50,854 [main] WARN  org.apache.pig.backend.hadoop.executionengine.map
ReduceLayer.MapReduceLauncher - Unable to retrieve job to compute warning aggregation.
2019-02-25 06:24:50,860 [main] INFO  org.apache.pig.backend.hadoop.executionengine.map
ReduceLayer.MapReduceLauncher - Success!
2019-02-25 06:24:50,870 [main] INFO  org.apache.pig.data.SchemaTupleBackend - Key [pig
.schematuple] was not set... will not generate code.
2019-02-25 06:24:50,996 [main] INFO  org.apache.hadoop.mapreduce.lib.input.FileInputFo
rmat - Total input paths to process : 1
2019-02-25 06:24:50,998 [main] INFO  org.apache.pig.backend.hadoop.executionengine.uti
l.MapRedUtil - Total input paths to process : 1
(2,8)
(7,8)
grunt>
```

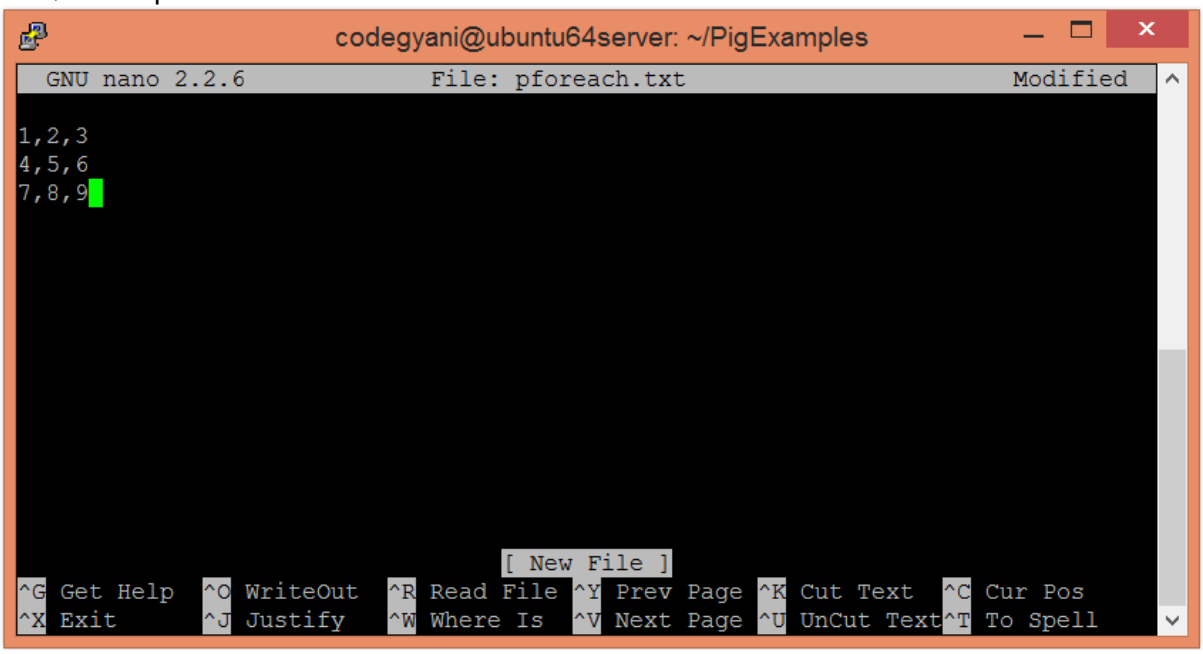
Apache Pig FOREACH Operator

The Apache Pig FOREACH operator generates data transformations based on columns of data. It is recommended to use FILTER operation to work with tuples of data.

Steps to execute FOREACH Operator

- Create a text file in your local machine and provide some values to it.

$ nano pforeach.txt



- Check the values written in the text files.

$ cat pforeach.txt

```
codegyani@ubuntu64server: ~/PigExamples
codegyani@ubuntu64server:~/PigExamples$ nano pforeach.txt
codegyani@ubuntu64server:~/PigExamples$ cat pforeach.txt
1,2,3
4,5,6
7,8,9
codegyani@ubuntu64server:~/PigExamples$
```

- Upload the text files on HDFS in the specific directory.

$ hdfs dfs -put pforeach.txt /pigexample

- Open the pig MapReduce run mode.

$ pig

- Load the file that contains the data.

```
grunt> A = LOAD '/pigexample/pforeach.txt' USING PigStorage(',')
                    AS (a1:int,a2:int,a3:int) ;
```

- Now, execute and verify the data.

grunt> DUMP A;

```
codegyani@ubuntu64server: ~/PigExamples
nect to server: 0.0.0.0/0.0.0.0:10020. Already tried 9 time(s); retry policy is
RetryUpToMaximumCountWithFixedSleep(maxRetries=10, sleepTime=1000 MILLISECONDS)
2019-02-25 09:45:40,166 [main] WARN  org.apache.pig.backend.hadoop.executionengi
ne.mapReduceLayer.MapReduceLauncher - Unable to retrieve job to compute warning
aggregation.
2019-02-25 09:45:40,166 [main] INFO  org.apache.pig.backend.hadoop.executionengi
ne.mapReduceLayer.MapReduceLauncher - Success!
2019-02-25 09:45:40,237 [main] INFO  org.apache.pig.data.SchemaTupleBackend - Ke
y [pig.schematuple] was not set... will not generate code.
2019-02-25 09:45:40,465 [main] INFO  org.apache.hadoop.mapreduce.lib.input.FileI
nputFormat - Total input paths to process : 1
2019-02-25 09:45:40,469 [main] INFO  org.apache.pig.backend.hadoop.executionengi
ne.util.MapRedUtil - Total input paths to process : 1
(1,2,3)
(4,5,6)
(7,8,9)
grunt>
```

- Let's traverse the data of two columns.

```
grunt> fe = FOREACH A generate a1,a2;
grunt> DUMP fe;
```

```
codegyani@ubuntu64server: ~/PigExamples
RetryUpToMaximumCountWithFixedSleep(maxRetries=10, sleepTime=1000 MILLISECONDS)
2019-02-25 09:55:54,433 [main] WARN  org.apache.pig.backend.hadoop.executionengi
ne.mapReduceLayer.MapReduceLauncher - Unable to retrieve job to compute warning
aggregation.
2019-02-25 09:55:54,440 [main] INFO  org.apache.pig.backend.hadoop.executionengi
ne.mapReduceLayer.MapReduceLauncher - Success!
2019-02-25 09:55:54,447 [main] INFO  org.apache.pig.data.SchemaTupleBackend - Ke
y [pig.schematuple] was not set... will not generate code.
2019-02-25 09:55:54,506 [main] INFO  org.apache.hadoop.mapreduce.lib.input.FileI
nputFormat - Total input paths to process : 1
2019-02-25 09:55:54,507 [main] INFO  org.apache.pig.backend.hadoop.executionengi
ne.util.MapRedUtil - Total input paths to process : 1
(1,2)
(4,5)
(7,8)
grunt>
```

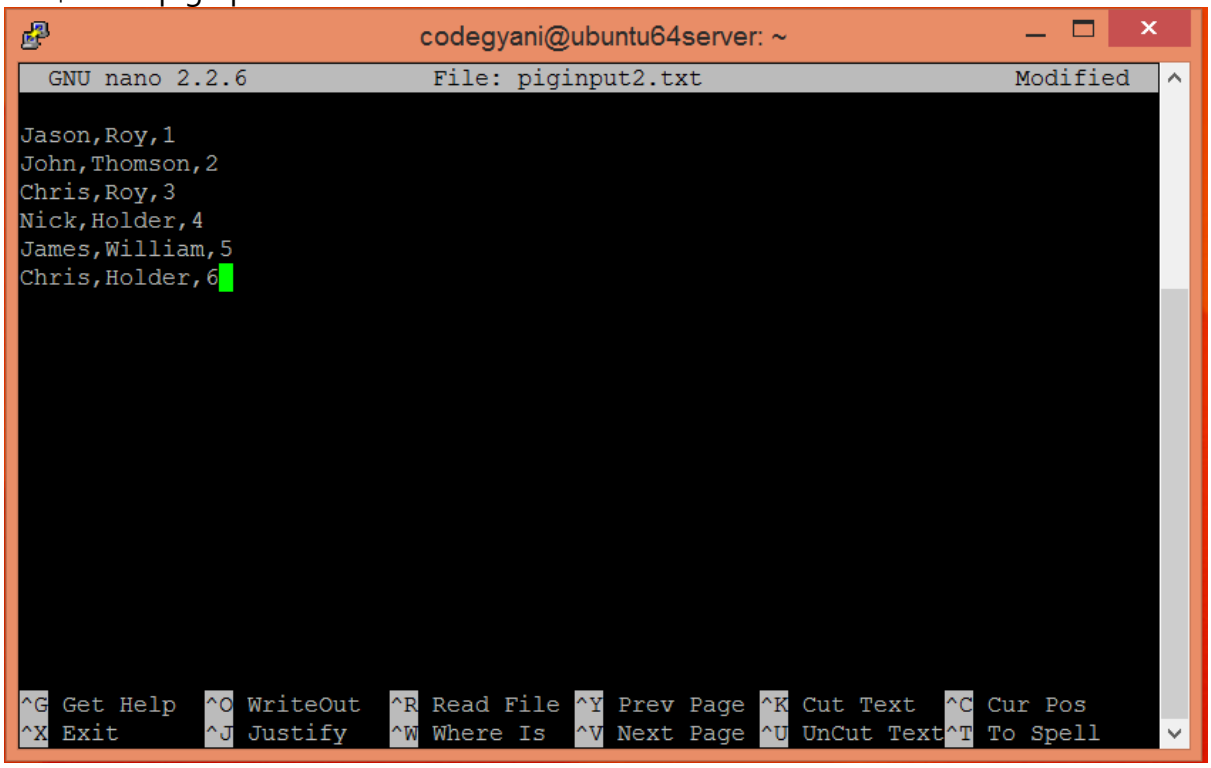
Apache Pig Group Operator

The Apache Pig GROUP operator is used to group the data in one or more relations. It groups the tuples that contain a similar group key. If the group key has more than one field, it treats as tuple otherwise it will be the same type as that of the group key. In a result, it provides a relation that contains one tuple per group.

Steps to execute Group Operator

- Create a text file in your local machine and write some text into it.

$ nano piginput2.txt



- Check the text written in the piginput2.txt file.

$ cat piginput2.txt

```
codegyani@ubuntu64server: ~
codegyani@ubuntu64server:~$ nano piginput2.txt
codegyani@ubuntu64server:~$ cat piginput2.txt
Jason,Roy,1
John,Thomson,2
Chris,Roy,3
Nick,Holder,4
James,William,5
Chris,Holder,6
codegyani@ubuntu64server:~$
```

- Upload the piginput2.txt file on HDFS in the specific directory.

$ hdfs dfs -put /home/codegyani/piginput2.txt /pigexample

Hadoop Overview Datanodes Snapshot Startup Progress Utilities

Browse Directory

/pigexample Go!

Permission	Owner	Group	Size	Last Modified	Replication	Block Size	Name
-rw-r--r--	codegyani	supergroup	31 B	2/25/2019, 2:10:07 AM	1	128 MB	piginput1.txt
-rw-r--r--	codegyani	supergroup	84 B	2/25/2019, 3:20:21 AM	1	128 MB	piginput2.txt

Hadoop, 2015.

- Open the pig MapReduce run mode.

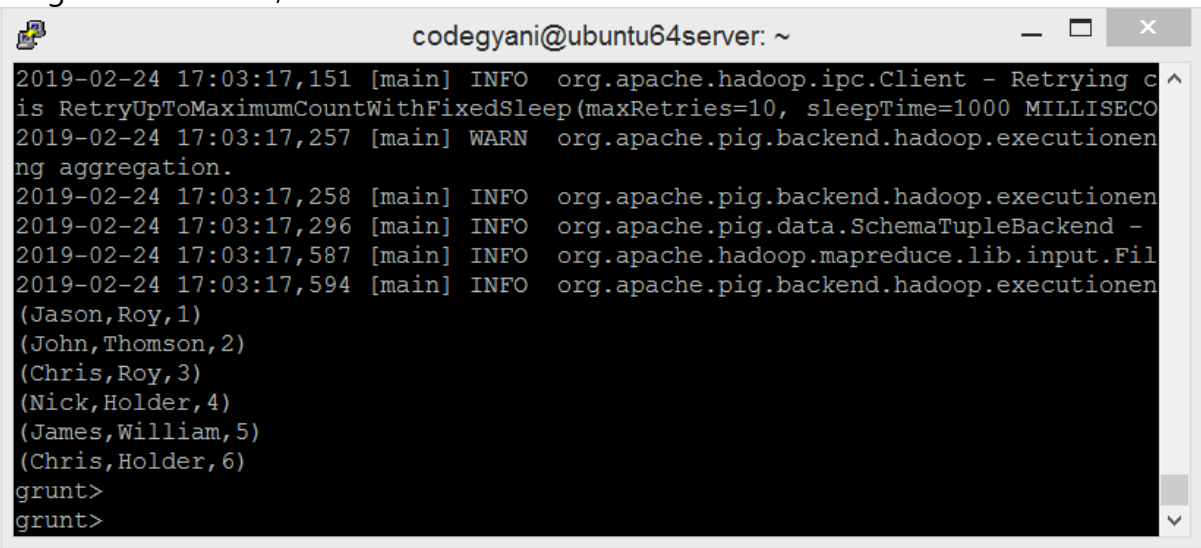
$ pig

- Load the data into the bag.

```
grunt> A = LOAD '/pigexample/piginput2.txt' USING PigStorage(',')
                                AS (fname:chararray,l_name:chararray,id:int);
```

- Now execute and verify the data.

```
grunt> DUMP A;
```



- Let us group the data on the basis of l_name.

```
grunt> group groupbylname = group A by l_name ;
```

- Now, execute and verify the data.

```
grunt> DUMP groupbylname;
```

```
codegyani@ubuntu64server: ~
is RetryUpToMaximumCountWithFixedSleep(maxRetries=10, sleepTime=1000 MILLISECO
2019-02-24 17:17:29,102 [main] INFO  org.apache.hadoop.ipc.Client - Retrying c
is RetryUpToMaximumCountWithFixedSleep(maxRetries=10, sleepTime=1000 MILLISECO
2019-02-24 17:17:30,106 [main] INFO  org.apache.hadoop.ipc.Client - Retrying c
is RetryUpToMaximumCountWithFixedSleep(maxRetries=10, sleepTime=1000 MILLISECO
2019-02-24 17:17:30,212 [main] WARN  org.apache.pig.backend.hadoop.executionen
ng aggregation.
2019-02-24 17:17:30,213 [main] INFO  org.apache.pig.backend.hadoop.executionen
2019-02-24 17:17:30,217 [main] INFO  org.apache.pig.data.SchemaTupleBackend -
2019-02-24 17:17:30,276 [main] INFO  org.apache.hadoop.mapreduce.lib.input.Fil
2019-02-24 17:17:30,281 [main] INFO  org.apache.pig.backend.hadoop.executionen
(Roy,{(Chris,Roy,3),(Jason,Roy,1)})
(Holder,{(Chris,Holder,6),(Nick,Holder,4)})
(Thomson,{(John,Thomson,2)})
(William,{(James,William,5)})
grunt>
```

```
(Roy,{(Chris,Roy,3),(Jason,Roy,1)})
(Holder,{(Chris,Holder,6),(Nick,Holder,4)})
(Thomson,{(John,Thomson,2)})
(William,{(James,William,5)})
```

```
Jason,Roy,1
John,Thomson,2
Chris,Roy,3
Nick,Holder,4
James,William,5
Chris,Holder,6
```

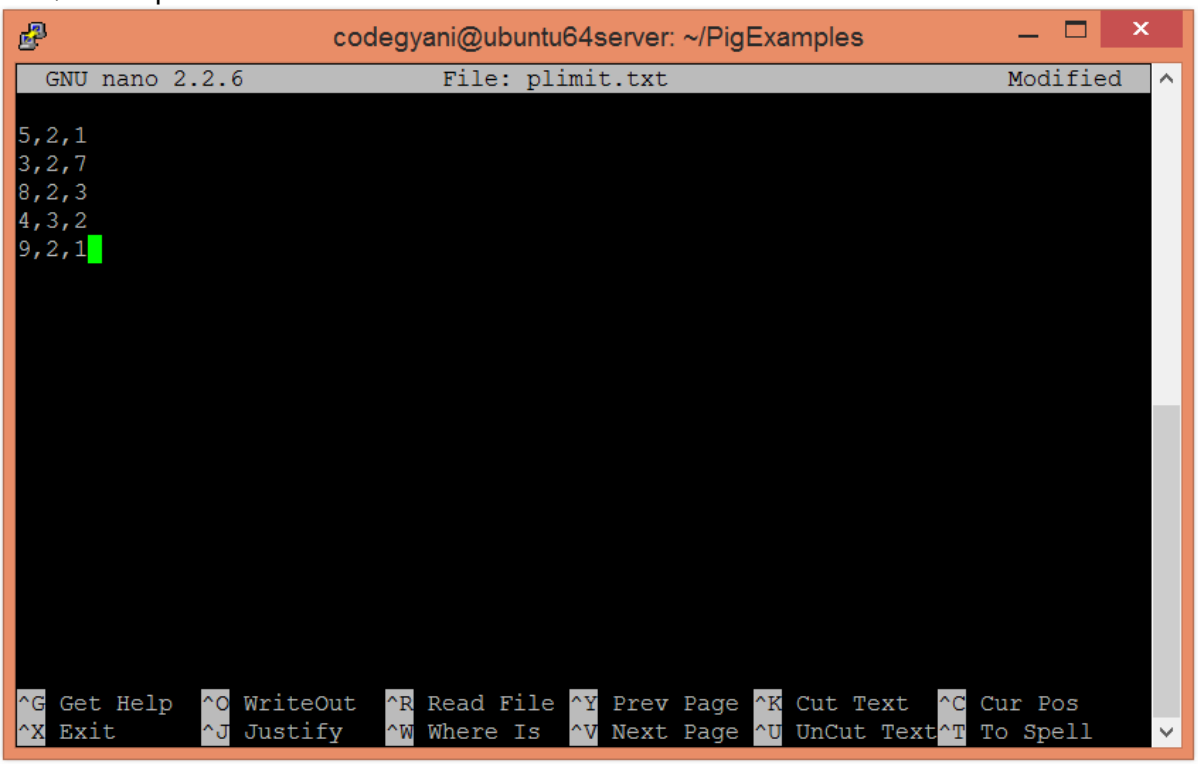
Apache Pig LIMIT Operator

The Apache Pig LIMIT operator is used to limit the number of output tuples. However, if you specify the limit of output tuples equal to or more than the number of tuples exists, all the tuples in the relation are returned.

Steps to execute LIMIT Operator

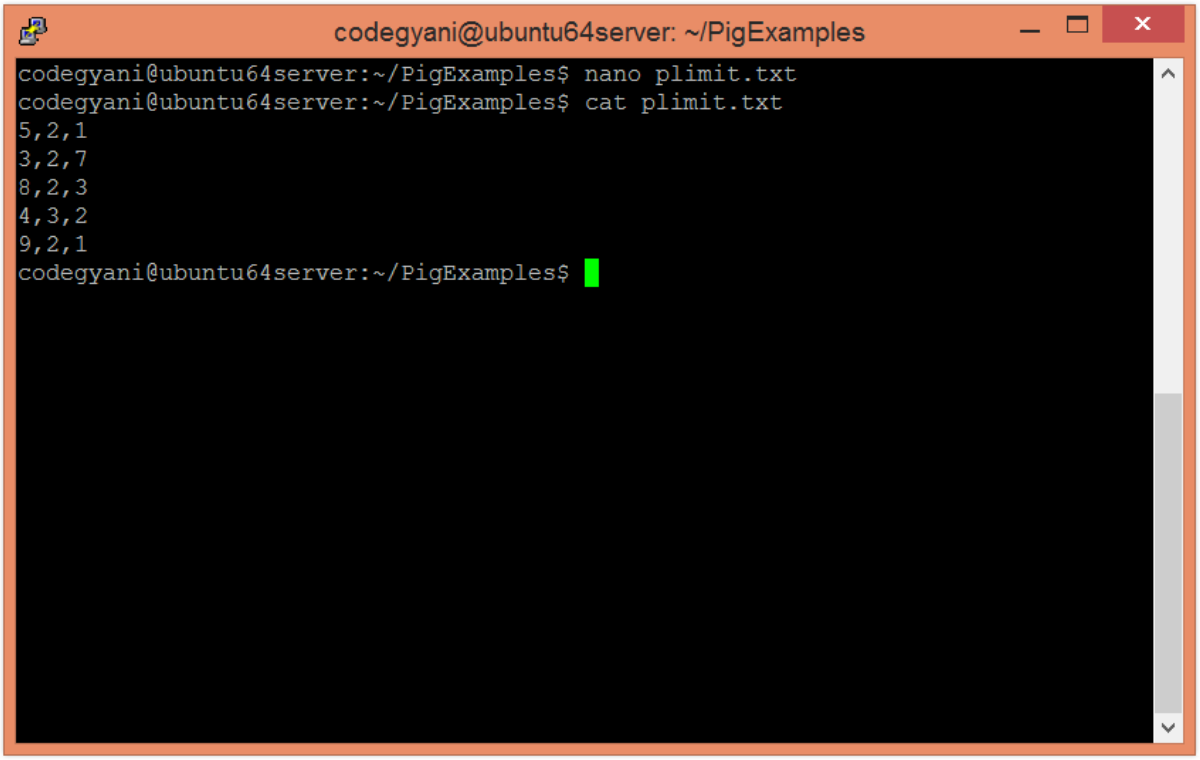
- Create a text file in your local machine and insert the list of tuples.

$ nano plimit.txt



- Check the tuples inserted in the text files.

$ cat plimit.txt



- Upload the text files on HDFS in the specific directory.

$ hdfs dfs -put plimit.txt /pigexample

- Open the pig MapReduce run mode.

$ pig

- Load the file that contains the data.

```
grunt> A = LOAD '/pigexample/plimit.txt' USING PigStorage(',')
                                AS (a1:int,a2:int,a3:int) ;
```

- Now, execute and verify the data.

grunt> DUMP A;

```
codegyani@ubuntu64server: ~/PigExamples
2019-02-25 13:29:08,383 [main] WARN  org.apache.pig.backend.hadoop.executionengi
ne.mapReduceLayer.MapReduceLauncher - Unable to retrieve job to compute warning
aggregation.
2019-02-25 13:29:08,388 [main] INFO  org.apache.pig.backend.hadoop.executionengi
ne.mapReduceLayer.MapReduceLauncher - Success!
2019-02-25 13:29:08,421 [main] INFO  org.apache.pig.data.SchemaTupleBackend - Ke
y [pig.schematuple] was not set... will not generate code.
2019-02-25 13:29:08,653 [main] INFO  org.apache.hadoop.mapreduce.lib.input.FileI
nputFormat - Total input paths to process : 1
2019-02-25 13:29:08,660 [main] INFO  org.apache.pig.backend.hadoop.executionengi
ne.util.MapRedUtil - Total input paths to process : 1
(5,2,1)
(3,2,7)
(8,2,3)
(4,3,2)
(9,2,1)
grunt>
```

- Let's return the first two tuples.

```
grunt> Result = LIMIT A 2;
grunt> DUMP Result;
```

```
codegyani@ubuntu64server: ~/PigExamples
2019-02-25 13:32:45,887 [main] INFO  org.apache.hadoop.mapreduce.lib.input.FileI
nputFormat - Total input paths to process : 1
2019-02-25 13:32:45,892 [main] INFO  org.apache.pig.backend.hadoop.executionengi
ne.util.MapRedUtil - Total input paths to process : 1
2019-02-25 13:32:46,744 [main] INFO  org.apache.hadoop.mapreduce.lib.output.File
OutputCommitter - Saved output of task 'attempt__0001_m_000001_1' to hdfs://192.
168.56.123:8020/tmp/temp-462791126/tmp-1745148875/_temporary/0/task__0001_m_0000
01
2019-02-25 13:32:46,974 [main] WARN  org.apache.pig.data.SchemaTupleBackend - Sc
hemaTupleBackend has already been initialized
2019-02-25 13:32:46,992 [main] INFO  org.apache.hadoop.mapreduce.lib.input.FileI
nputFormat - Total input paths to process : 1
2019-02-25 13:32:46,994 [main] INFO  org.apache.pig.backend.hadoop.executionengi
ne.util.MapRedUtil - Total input paths to process : 1
(5,2,1)
(3,2,7)
grunt>
```

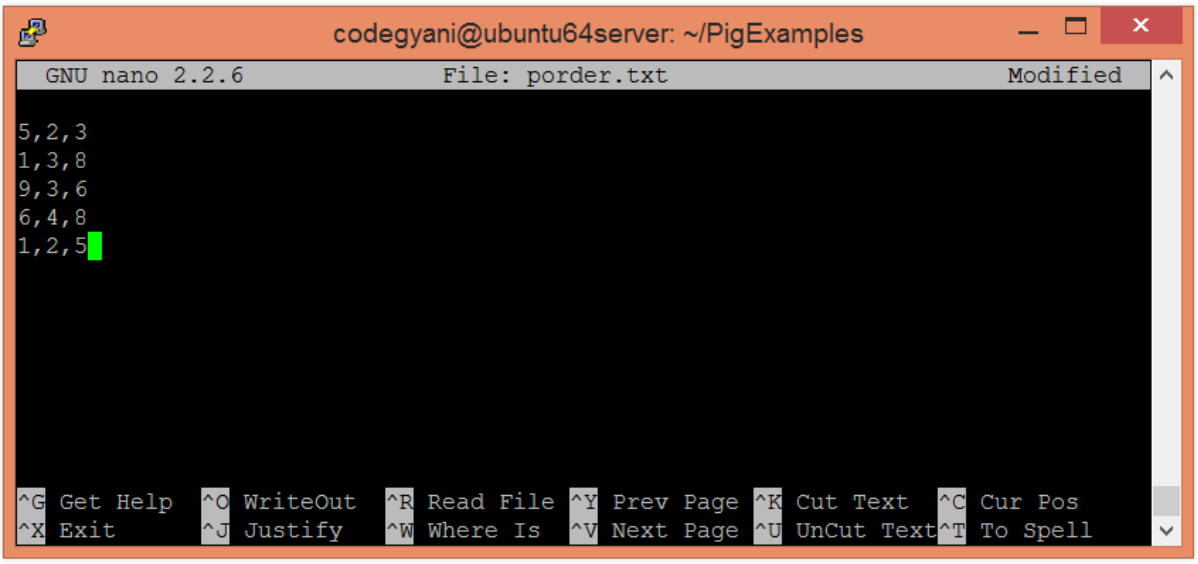
Apache Pig ORDER BY Operator

The Apache Pig ORDER BY operator sorts a relation based on one or more fields. It maintains the order of tuples.

Steps to execute ORDER BY Operator

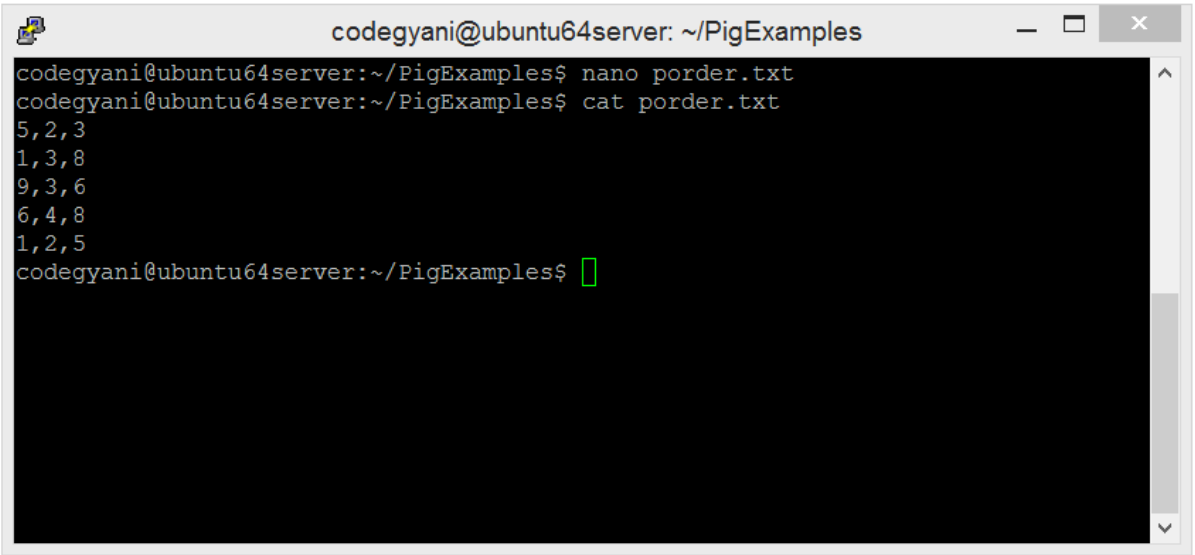
- Create a text file in your local machine and insert the list of tuples.

$ nano porder.txt



- Check the tuples inserted in the text files.

$ cat porder.txt



- Upload the text files on HDFS in the specific directory.

$ hdfs dfs -put porder.txt /pigexample

- Open the pig MapReduce run mode.

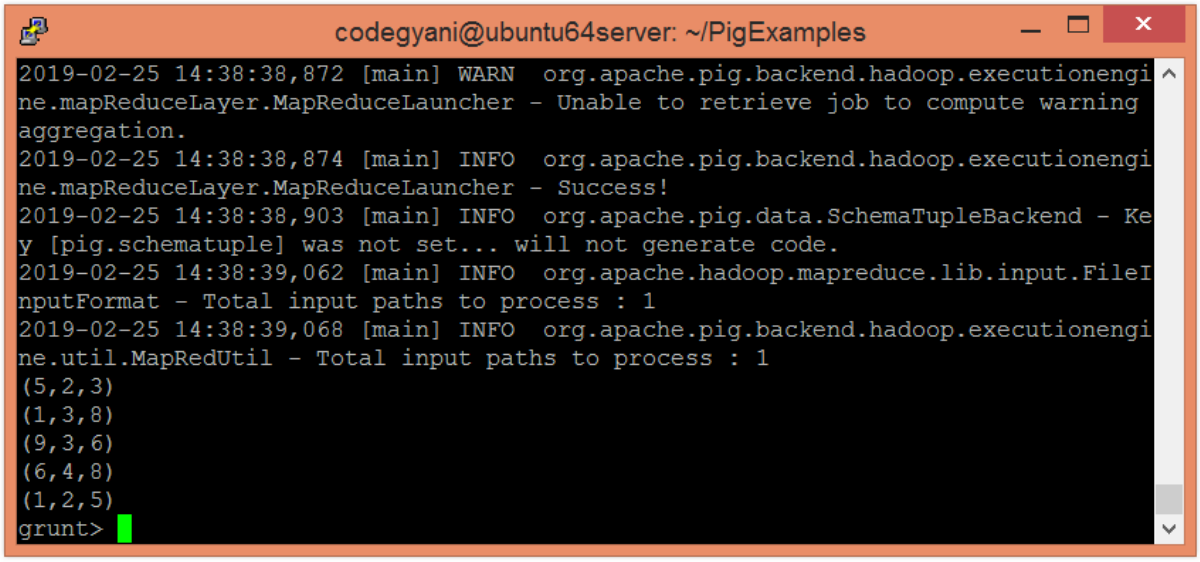
$ pig

- Load the file that contains the data.

```
grunt> A = LOAD '/pigexample/porder.txt' USING PigStorage(',')
                    AS (a1:int,a2:int,a3:int) ;
```

- Now, execute and verify the data.

```
grunt> DUMP A;
```



- Let's return the first two tuples.

```
grunt> Result = ORDER A BY a1 DESC;
grunt> DUMP Result;
```

codegyani@ubuntu64server: ~/PigExamples

```
2019-02-25 14:57:12,386 [main] WARN  org.apache.pig.backend.hadoop.executionengi
ne.mapReduceLayer.MapReduceLauncher - Unable to retrieve job to compute warning
aggregation.
2019-02-25 14:57:12,391 [main] INFO  org.apache.pig.backend.hadoop.executionengi
ne.mapReduceLayer.MapReduceLauncher - Success!
2019-02-25 14:57:12,420 [main] INFO  org.apache.pig.data.SchemaTupleBackend - Ke
y [pig.schematuple] was not set... will not generate code.
2019-02-25 14:57:12,496 [main] INFO  org.apache.hadoop.mapreduce.lib.input.FileI
nputFormat - Total input paths to process : 1
2019-02-25 14:57:12,497 [main] INFO  org.apache.pig.backend.hadoop.executionengi
ne.util.MapRedUtil - Total input paths to process : 1
(9,3,6)
(6,4,8)
(5,2,3)
(1,2,5)
(1,3,8)
grunt>
```

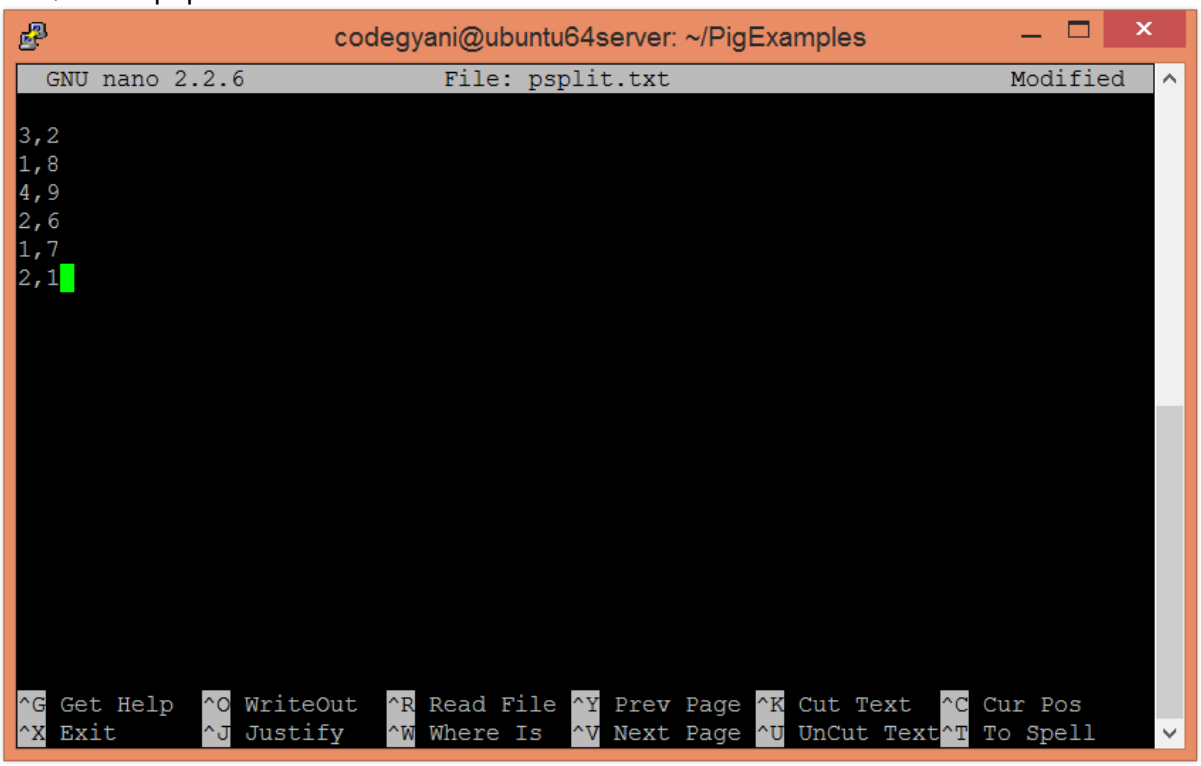
Apache Pig SPLIT Operator

The Apache Pig SPLIT operator breaks the relation into two or more relations according to the provided expression. Here, a tuple may or may not be assigned to one or more than one relation.

Steps to execute SPLIT Operator

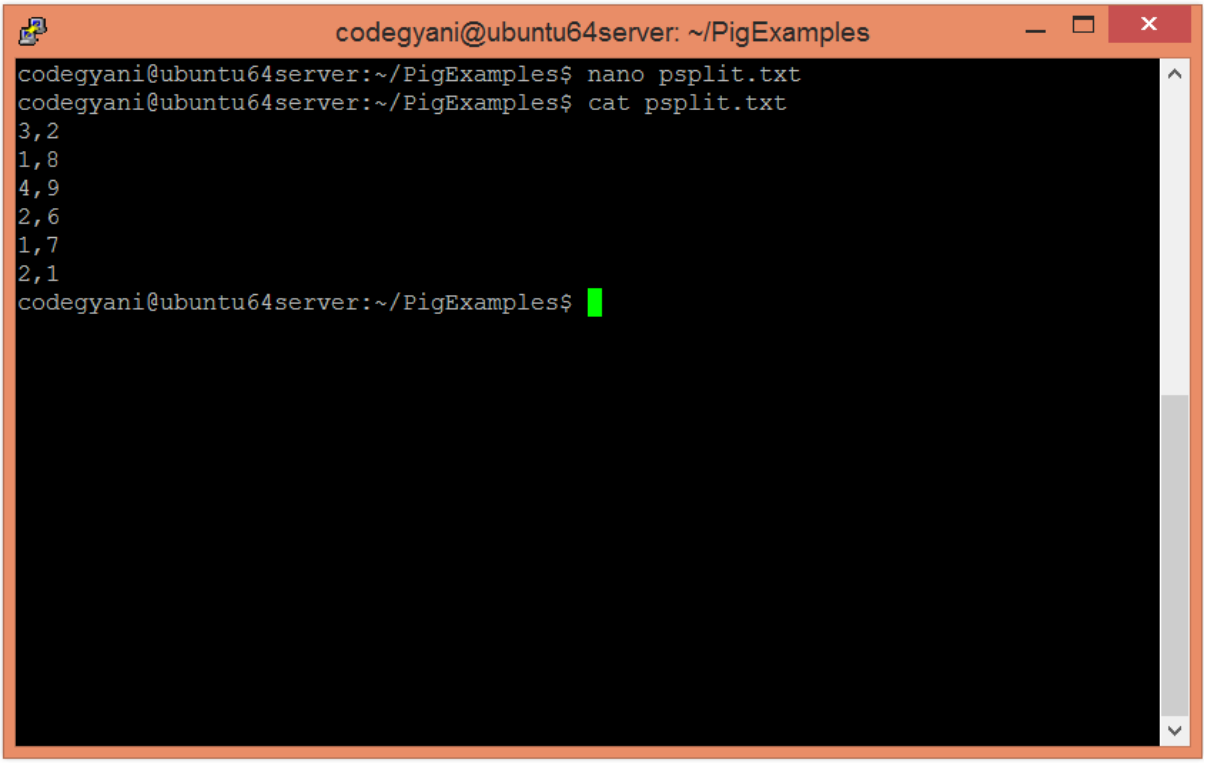
- Create a text file in your local machine and provide some values to it.

$ nano psplit.txt



- Check the values written in the text files.

$ cat psplit.txt



- Upload the text files on HDFS in the specific directory.

$ hdfs dfs -put psplit.txt /pigexample

- Open the pig MapReduce run mode.

$ pig

- Load the file that contains the data.

```
grunt> A = LOAD '/pigexample/psplit.txt' USING PigStorage(',') AS (a1:int,a2:int) ;
```

- Now, execute and verify the data.

```
grunt> DUMP A;
```

```
codegyani@ubuntu64server: ~/PigExamples
aggregation.
2019-02-25 17:42:30,274 [main] INFO  org.apache.pig.backend.hadoop.executionengi
ne.mapReduceLayer.MapReduceLauncher - Success!
2019-02-25 17:42:30,311 [main] INFO  org.apache.pig.data.SchemaTupleBackend - Ke
y [pig.schematuple] was not set... will not generate code.
2019-02-25 17:42:30,649 [main] INFO  org.apache.hadoop.mapreduce.lib.input.FileI
nputFormat - Total input paths to process : 1
2019-02-25 17:42:30,657 [main] INFO  org.apache.pig.backend.hadoop.executionengi
ne.util.MapRedUtil - Total input paths to process : 1
(3,2)
(1,8)
(4,9)
(2,6)
(1,7)
(2,1)
grunt>
```

- Let's provide the expression to split the relation.

grunt> SPLIT A INTO X IF a1<=2, Y IF a1>2;

```
codegyani@ubuntu64server: ~/PigExamples
aggregation.
2019-02-25 17:53:37,272 [main] INFO  org.apache.pig.backend.hadoop.executionengi
ne.mapReduceLayer.MapReduceLauncher - Success!
2019-02-25 17:53:37,277 [main] INFO  org.apache.pig.data.SchemaTupleBackend - Ke
y [pig.schematuple] was not set... will not generate code.
2019-02-25 17:53:37,407 [main] INFO  org.apache.hadoop.mapreduce.lib.input.FileI
nputFormat - Total input paths to process : 1
2019-02-25 17:53:37,410 [main] INFO  org.apache.pig.backend.hadoop.executionengi
ne.util.MapRedUtil - Total input paths to process : 1
(1,8)
(2,6)
(1,7)
(2,1)
grunt>
```

- Now, execute and verify the data of the first relation.

grunt> DUMP X;

```
codegyani@ubuntu64server: ~/PigExamples
aggregation.
2019-02-25 17:53:37,272 [main] INFO  org.apache.pig.backend.hadoop.executionengi
ne.mapReduceLayer.MapReduceLauncher - Success!
2019-02-25 17:53:37,277 [main] INFO  org.apache.pig.data.SchemaTupleBackend - Ke
y [pig.schematuple] was not set... will not generate code.
2019-02-25 17:53:37,407 [main] INFO  org.apache.hadoop.mapreduce.lib.input.FileI
nputFormat - Total input paths to process : 1
2019-02-25 17:53:37,410 [main] INFO  org.apache.pig.backend.hadoop.executionengi
ne.util.MapRedUtil - Total input paths to process : 1
(1,8)
(2,6)
(1,7)
(2,1)
grunt>
```

- Now, execute and verify the data of the second relation.

grunt> DUMP Y;

```
codegyani@ubuntu64server: ~/PigExamples
2019-02-25 18:01:04,227 [main] WARN  org.apache.pig.backend.hadoop.executionengi
ne.mapReduceLayer.MapReduceLauncher - Unable to retrieve job to compute warning
aggregation.
2019-02-25 18:01:04,232 [main] INFO  org.apache.pig.backend.hadoop.executionengi
ne.mapReduceLayer.MapReduceLauncher - Success!
2019-02-25 18:01:04,234 [main] INFO  org.apache.pig.data.SchemaTupleBackend - Ke
y [pig.schematuple] was not set... will not generate code.
2019-02-25 18:01:04,314 [main] INFO  org.apache.hadoop.mapreduce.lib.input.FileI
nputFormat - Total input paths to process : 1
2019-02-25 18:01:04,321 [main] INFO  org.apache.pig.backend.hadoop.executionengi
ne.util.MapRedUtil - Total input paths to process : 1
(3,2)
(4,9)
grunt>
```

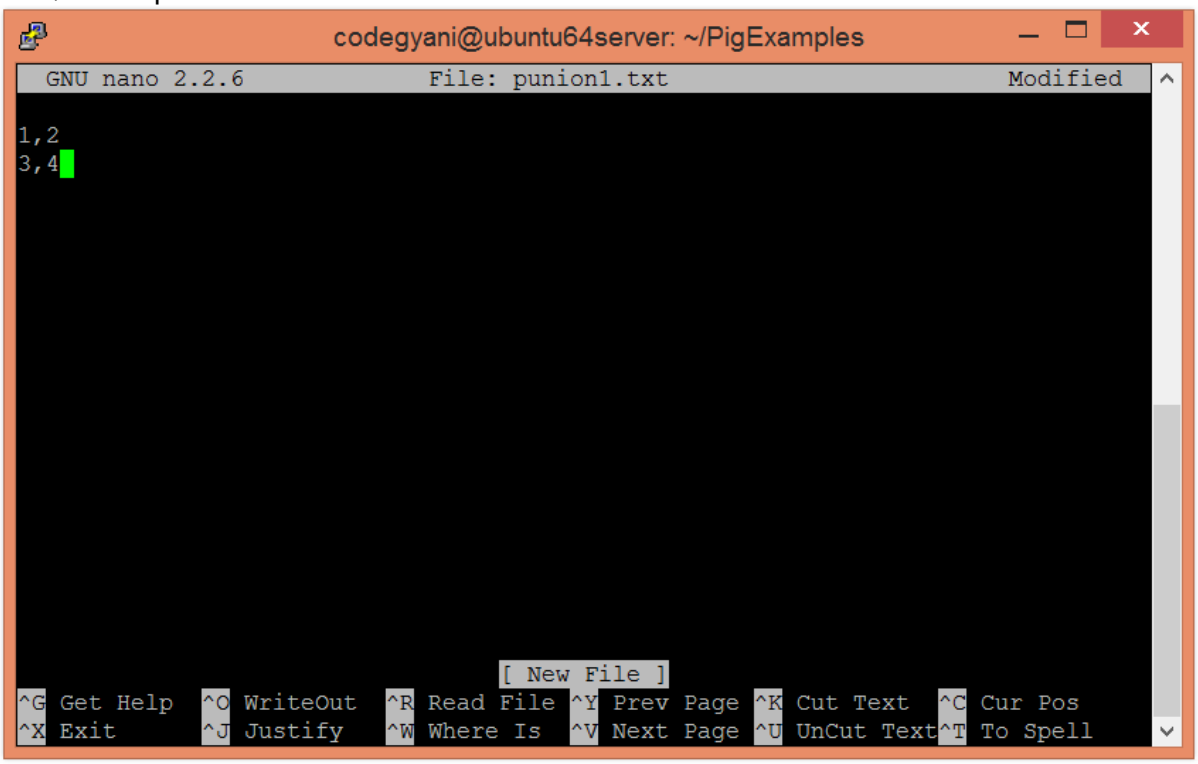
Apache Pig UNION Operator

The Apache Pig UNION operator is used to compute the union of two or more relations. It doesn't maintain the order of tuples. It also doesn't eliminate the duplicate tuples.

Steps to execute UNION Operator

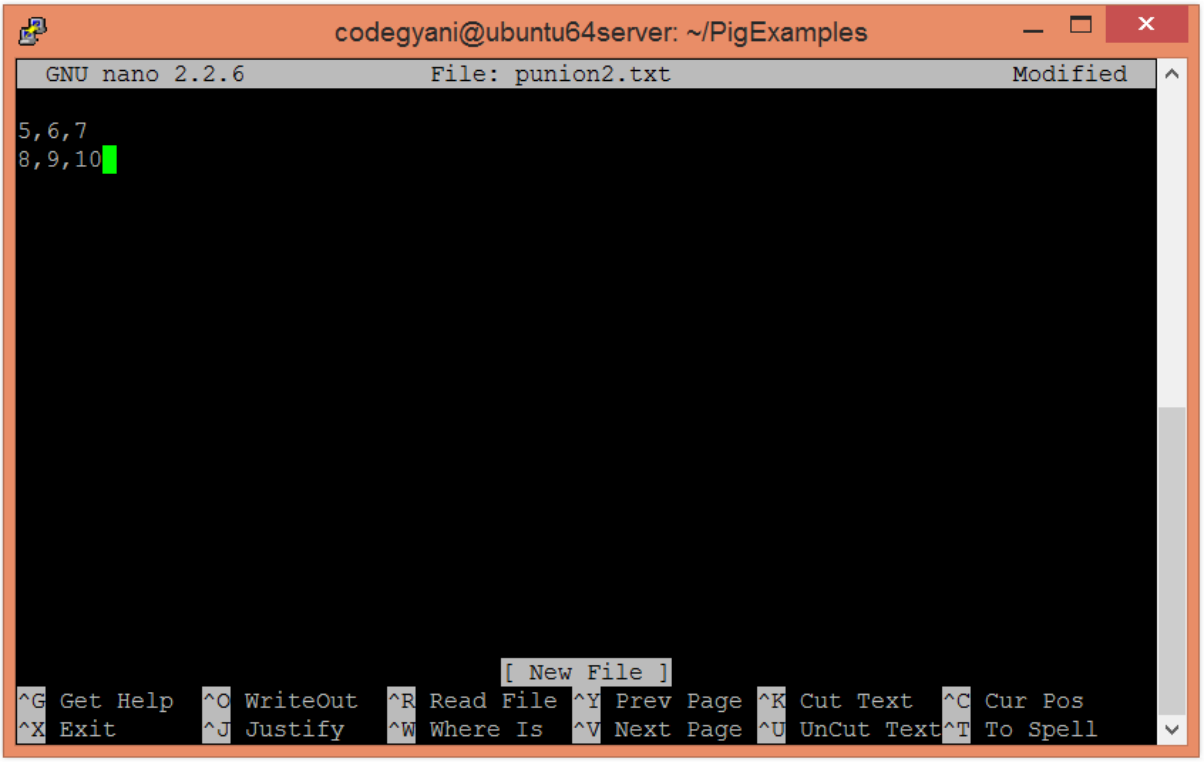
- Create a text file in your local machine and write some values into it.

$ nano punion1.txt



- Create another text file in your local machine and also write some values into it.

$ nano punion2.txt



- Check the values written in both text files.

$ cat punion1.txt

$ cat punion2.txt

```
codegyani@ubuntu64server: ~/PigExamples
codegyani@ubuntu64server:~/PigExamples$ cat punion1.txt
1,2
3,4
codegyani@ubuntu64server:~/PigExamples$ cat punion2.txt
5,6,7
8,9,10
codegyani@ubuntu64server:~/PigExamples$
```

- Upload both text files on HDFS in the specific directory.

$ hdfs dfs -put punion1.txt /pigexample

$ hdfs dfs -put punion2.txt /pigexample

- Open the pig MapReduce run mode.

$ pig

- Load the file that contains the data.

grunt> A = load '/pigexample/punion1.txt' using PigStorage(',') as (a1:int,a2:int);

- Now, execute and verify the data.

grunt> DUMP A;

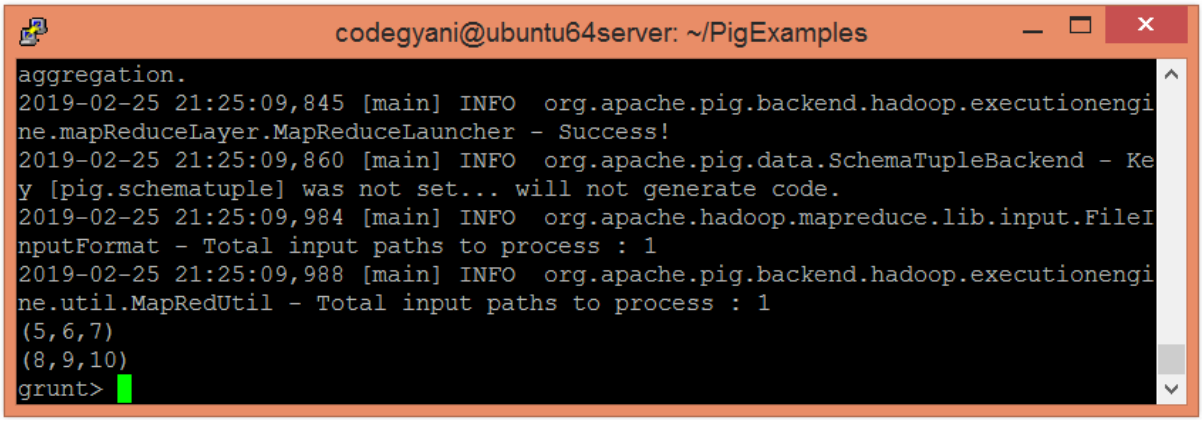
```
codegyani@ubuntu64server: ~/PigExamples
aggregation.
2019-02-25 21:18:54,655 [main] INFO  org.apache.pig.backend.hadoop.executionengi
ne.mapReduceLayer.MapReduceLauncher - Success!
2019-02-25 21:18:54,686 [main] INFO  org.apache.pig.data.SchemaTupleBackend - Ke
y [pig.schematuple] was not set... will not generate code.
2019-02-25 21:18:54,866 [main] INFO  org.apache.hadoop.mapreduce.lib.input.FileI
nputFormat - Total input paths to process : 1
2019-02-25 21:18:54,873 [main] INFO  org.apache.pig.backend.hadoop.executionengi
ne.util.MapRedUtil - Total input paths to process : 1
(1,2)
(3,4)
grunt>
```

- Load the another file that contains the data.

```
grunt> B = LOAD '/pigexample/punion2.txt' USING PigStorage(',')
                    AS (b1:int,b2:int,b3:int);
```

- Now, execute and verify the data.

```
grunt> DUMP B;
```

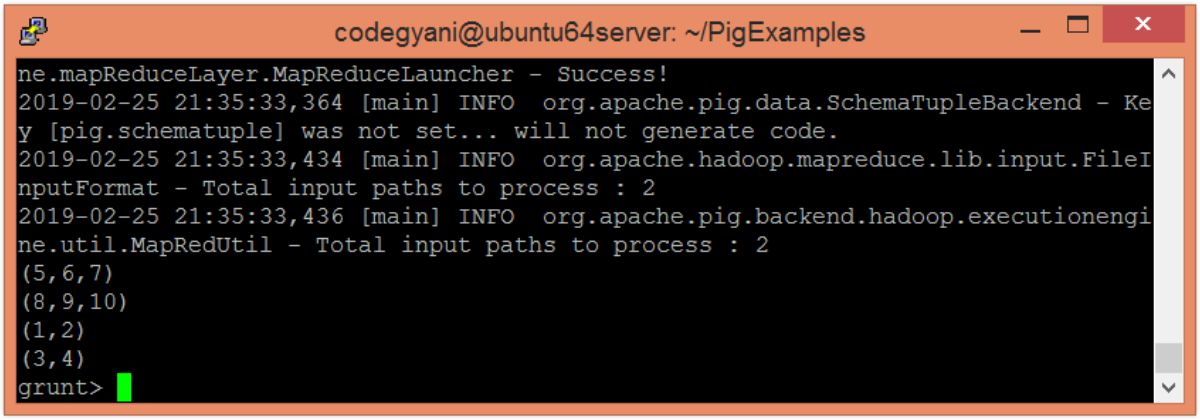


- Let's perform the union operation between both files.

```
grunt> Result = UNION A,B;
```

- Now, execute and verify the data.

```
grunt> DUMP Result;
```



Parallelism

When running in MapReduce mode, it's important that the degree of parallelism matches the size of the dataset. By default, Pig sets the number of reducers by looking at the size of the input and using one reducer per 1 GB of input, up to a maximum of 999 reducers.

pig.exec.reducers.bytes.per.reducer(the default is 1,000,000,000 bytes)

pig.exec.reducers.max(the default is 999)

To explicitly set the number of reducers you want for each job, you can use a PARALLEL clause for operators that run in the reduce phase. These include all the grouping and joining operators (GROUP, COGROUP, JOIN, CROSS), as well as DISTINCT and ORDER.

Thefollowing line sets the number of reducers to 30 for the GROUP

grouped_records = **GROUP** records **BY** year **PARALLEL 30**;

Alternatively, you can set the default_paralleloption, and it will take effect for all subsequent jobs:

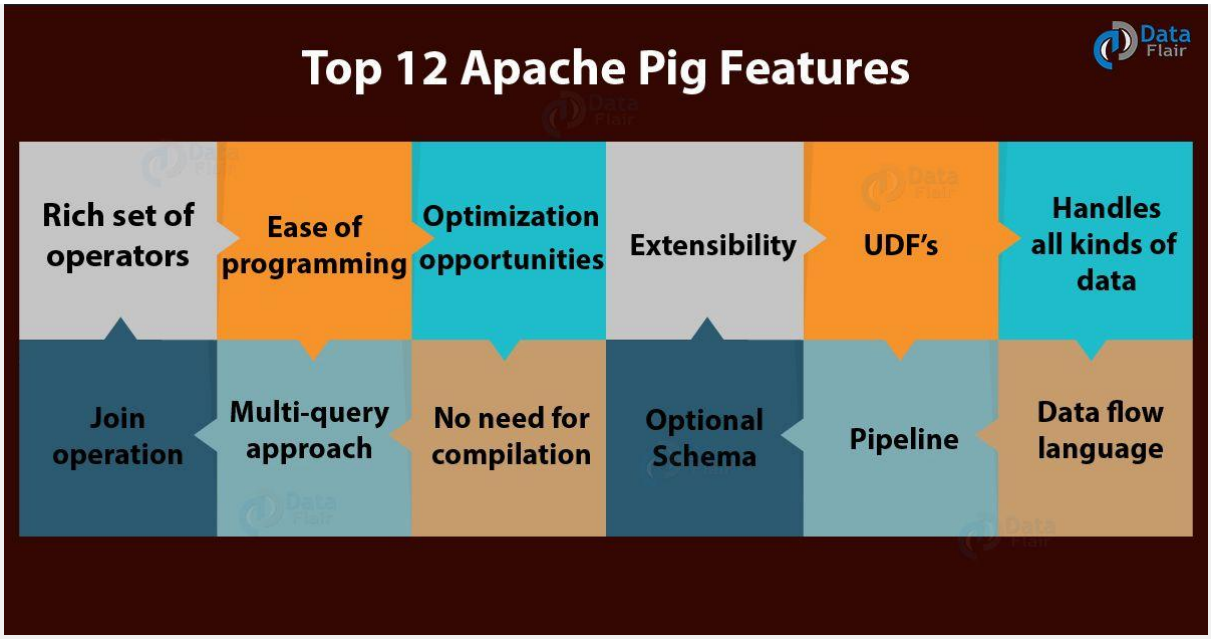
grunt>**set default_parallel 30**

The number of map tasks is set by the size of the input (with one map per HDFS block) and is not affected by the PARALLEL clause.

Top 12 Apache Pig Features You Must Know

After learning the complete **Introduction of Apache Pig**, in this article, we will discuss the 12 best features of Apache Pig. There are many features of Apache Pig such as Ease of programming, Handles all kinds of data, Extensibility and many more.

So, in this blog, “Apache Pig Features” we will discuss all these features in detail and try to understand why Pig should be chosen.



Top 12 Apache Pig Features You Must Know

Top 12 Hadoop Pig Features

There are lot many Apache Pig features. Let’s discuss them one by one:

i. Rich set of operators

One of the major advantages is, in order to perform several operations, there is a huge set of operators offered by Apache Pig, such as join, sort, filer, etc.

ii. Ease of programming

Basically, for SQL Programmer, Pig Latin is a boon. It is as similar to SQL. Hence, if you are good at SQL it is easy to write a Pig script.

iii. Optimization opportunities

Also, it’s a benefit working here because in Apache Pig the tasks optimize their execution automatically. Hence, as a result, programmers only need to focus on the semantics of the language.

iv. Extensibility

Extensibility is one of the most interesting features it has. It means users can develop their own functions to read, process, and write data, using the existing operators.

v. UDF's

It is also a very amazing feature that it offers the facility to create User-defined Functions in other programming languages like Java. Meanwhile, invoke or embed them in Pig Scripts.

vi. Handles all kinds of data

Handling all kinds of data is one of the reasons for easy programming. That means it analyzes all kinds of data. Either structured or unstructured. Also, it stores the results in **HDFS**.

vii. Join operation

In Apache Pig, performing a Join operation is pretty simple.

viii. Multi-query approach

Apache Pig uses multi-query approach. Basically, this reduces the length of the codes to a great extent.

ix. No need for compilation

Here, we do not require any compilation. Since every Apache Pig operator is converted internally into a MapReduce job on execution.

x. Optional Schema

However, the schema is optional, in Apache Pig. Hence, without designing a schema we can store data. So, values are stored as $01, $02 etc.

xi. Pipeline

Apache Pig Latin allows splits in the pipeline

xii. Data flow language

Apache Pig is data flow language.